

ĐÁNH GIÁ, ĐỘ TIN CẬY VÀ DỰ ÁN

TÓM TẮT

Phần E trả lời câu hỏi khó chịu nhất: làm sao biết mình không đang tự lừa mình. Bốn chương gồm đọc paper, thiết kế thí nghiệm và ablation, độ tin cậy và vận hành, và dự án xuyên suốt qua ba vòng. Đọc xong bạn phân biệt được một cải thiện thật với một cải thiện nằm trong khoảng nhiễu, và biết một so sánh không công bằng trông như thế nào. Nếu chỉ nhớ một điều: cơ chế tự lừa trong học máy rất cụ thể và xảy ra với người trung thực đang vội, nên nó phải được chặn bằng quy trình chứ không bằng thiện chí.

Bốn phần trước dạy cách xây. Phần này trả lời câu hỏi khó chịu hơn và quan trọng hơn: **làm sao biết mình không đang tự lừa mình?** Đó không phải một câu hỏi về đạo đức mà là một câu hỏi kỹ thuật, bởi vì cơ chế tự lừa trong học máy rất cụ thể và đã được mô tả kỹ: một baseline chưa được tune tử tế, một test set đã bị nhìn quá nhiều lần, một cải thiện nằm gọn trong khoảng nhiễu giữa các lần chạy, một ablation chỉ chạy một hạt giống. Không cái nào trong số đó cần ác ý; tất cả đều xảy ra với người trung thực đang vội.

Chương 23 là kỹ năng đọc: đọc paper theo ba lượt, đọc phần giới hạn trước phần kết quả, và ghi chú theo cách buộc phải nêu được chỗ mình không tin. Chương 24 là chương nặng nhất của phần và là nơi phần lớn giá trị nằm: thiết kế so sánh cho công bằng, kỹ thuật ablation, lượng thống kê tối thiểu để không kết luận sai, phân tích lỗi có hệ thống, và cách đọc ablation của người khác. Chương 25 chuyển từ đúng sang *đáng tin*: trực quan hoá và những gì nó nói dối, hiệu chỉnh xác suất, ước lượng bất định, rồi vận hành sau khi mô hình đã ra thế giới. Chương 26 đóng vòng bằng dự án xuyên suốt qua ba vòng lặp, và chứa luôn thứ tự học đề xuất cho cả cây.

Một điểm đáng nói trước khi vào: kết luận âm — một ý tưởng đã thử và không có tác dụng — vẫn là kết quả, và là kết quả tồi tệ nhất khi bị mất. Ghi nó lại đầy đủ ngang với kết quả dương là một trong những thói quen phân biệt người làm nghiên cứu tử tế với người sưu tầm số đẹp.

Bảng 1. Bản đồ Phần E: các chương, nội dung, và mức đầu tư đề xuất.

Ch.	Thư mục	Nội dung	Mức
23	23-doc-paper	Đọc ba lượt; đọc giới hạn trước kết quả; ghi chú phải nêu chỗ mình không tin	cốt lõi
24	24-thi-nghiem-va-ablati	Thiết kế so sánh, ablation, thống kê tối thiểu, phân tích lỗi, báo cáo	cốt lõi
25	25-do-tin-cay-va-van-ha	Trực quan hoá và giới hạn của nó, hiệu chỉnh, bất định, vận hành	cốt lõi
26	26-lo-trinh-hoc	Ba vòng dự án, gợi ý đề tài, và thứ tự học đề xuất cho cả cây	cốt lõi

Chương 23 và 24 nên đọc song song với phần C chứ không đợi tới cuối, vì chúng thay đổi cách bạn đọc mọi paper trong phần C. Chương 26 chứa thứ tự học đề xuất cho toàn cây.

Đọc paper và theo dõi một lĩnh vực (Reading Papers, Following a Field)

PHỤ THUỘC B/03-giai-phau-he-thong (ba tầng bài toán → nguyên lý → cài đặt); B/07-chia-du-lieu-va-danh-gia (giao thức đánh giá — cần để đọc được phần dữ liệu của paper). Đọc song song với E/24-thi-nghiem-va-ablation: chương này dạy *đọc bằng chứng*, chương 24 dạy *tạo ra bằng chứng*, và hai kỹ năng đó là hai mặt của cùng một tấm giấy.

TÓM TẮT

Chương này là về cách phân bổ sự chú ý khi số paper vượt xa thời gian đọc, và về cách tách đóng góp thật khỏi cách kể chuyện. Đọc xong bạn định vị được một paper lạ trong bốn mươi phút. Bạn phát biểu được đóng góp thật nằm ở đâu, bằng chứng nào chưa đủ, và thí nghiệm nào sẽ khiến bạn đổi ý. Chừng đó đủ để quyết định có bỏ hai tuần tái hiện nó hay không. Hai thao tác đối nhiều nhất là đọc limitations và phần dữ liệu *trước* bảng kết quả, và ghi lại mục “chỗ tôi không tin” kèm thí nghiệm sẽ thuyết phục được mình. Nếu chỉ nhớ một điều: paper là văn bản thuyết phục chứ không phải biên bản thí nghiệm, nên câu hỏi trung tâm luôn là *bỏ đóng góp chính đi nhưng giữ nguyên công thức huấn luyện thì còn lại bao nhiêu*.

KIẾN THỨC CẦN CÓ TRƯỚC

- **paper** — một bài báo khoa học, thường 8–10 trang, có cấu trúc cố định: abstract, mở đầu, phương pháp, thí nghiệm, kết luận. “Preprint” là bản tự đăng (thường trên arXiv) chưa qua bình duyệt.
- **baseline** — phương pháp đối chứng mà kết quả mới phải vượt qua. Một baseline chưa được tinh chỉnh tử tế làm mọi so sánh sau đó vô nghĩa, nên chất lượng baseline quan trọng ngang chất lượng phương pháp mới.
- **ablation** — thí nghiệm bỏ đi hoặc thay thế đúng một thành phần của hệ thống rồi đo lại, để biết thành phần đó đóng góp bao nhiêu. Đây là bằng chứng chính cho câu “vì sao nó hoạt động”.
- **công thức huấn luyện (recipe)** — toàn bộ các lựa chọn quanh việc huấn luyện mà không phải kiến trúc: số epoch, learning rate và lịch của nó, augmentation, cách khởi tạo, EMA trọng số. Đối recipe một mình đã đủ tạo ra khác biệt lớn về kết quả.
- **seed** — số khởi tạo cho bộ sinh số ngẫu nhiên. Đổi seed thì khởi tạo trọng số và thứ tự dữ liệu đổi theo, nên cùng một cấu hình chạy hai seed vẫn ra hai con số khác nhau.
- **tập validation và tập test** — validation là tập dùng để *chọn* cấu hình; test là tập chỉ được dùng một lần để *báo cáo*. Trộn hai vai này là dạng rò rỉ phổ biến nhất trong các bảng kết quả.
- **mAP** — *mean average precision* — metric tổng hợp thông dụng cho phát hiện vật thể, gộp cả độ chính xác lẫn độ bao phủ qua nhiều ngưỡng. Ở đây bạn chỉ cần biết nó là “điểm số càng cao càng tốt”, đơn vị thường tính theo điểm phần trăm.
- **benchmark** — một tập dữ liệu cộng giao thức đánh giá cố định (ví dụ COCO, ADE20K) để nhiều nhóm so được với nhau. Con số chỉ có nghĩa khi kèm tên benchmark và phiên bản giao thức.

CÂU HỎI MỞ ĐẦU

- Q1. Vì sao đọc phần giới hạn của một paper *trước* bảng kết quả lại tiết kiệm thời gian hơn, dù lượng chữ bạn nạp vào là y hệt?

- Q2. Hai paper cùng báo cáo +1,0 điểm trên cùng benchmark; một cái đáng tin, một cái không. Chi tiết nào trong phần thiết lập tách được chúng, và vì sao nó không nằm trong bảng kết quả?
- Q3. Nếu phần lớn “tiền bộ” của một mảng biến mất khi baseline được tinh chỉnh bằng cùng ngân sách, thì suốt cả thập kỷ các paper đó đã thật sự đo cái gì?
- Q4. Ghi chú tóm tắt một paper gần như vô dụng sau sáu tháng. Mục nào trong ghi chú thì không, và vì sao đúng mục đó lại làm kỹ năng đọc của bạn lên tay?

1.1 Đặt vấn đề

Một mảng hẹp của thị giác máy tính hiện xuất bản nhiều hơn số paper mà một người đọc kỹ được trong cả năm. Bài toán vì thế không phải là “đọc thế nào cho hiểu” — điều đó bạn làm được với bất kỳ paper đơn lẻ nào nếu có đủ thời gian — mà là “phân bổ sự chú ý thế nào để không phá sản”. Phần lớn người tự học giải bài toán này sai theo cùng một cách: đọc tuần tự từ đầu đến cuối mọi paper thấy thú vị, đọc chậm dần, rồi bỏ cuộc và chuyển sang chỉ đọc abstract trên mạng xã hội. Cả hai cực đều không cho ra hiểu biết dùng được.

Có một lý do sâu hơn khiến việc đọc phải có kỹ thuật riêng. Paper là **một văn bản thuyết phục, không phải một biên bản thí nghiệm**. Tác giả đã biết kết quả trước khi viết câu đầu tiên của phần mở đầu, nên toàn bộ trật tự trình bày được tối ưu ngược từ bảng kết quả trở lên: đóng góp được đặt tên sao cho nghe như một nguyên lý, các lựa chọn không hiệu quả biến mất, và những lần chạy hỏng không xuất hiện ở đâu cả. Điều này không đòi hỏi ai phải gian dối; nó là hệ quả tự nhiên của việc một cộng đồng thưởng cho câu chuyện gọn gàng. Do đó câu hỏi trung tâm khi đọc không phải “họ làm gì” mà là **bằng chứng có tương xứng với tuyên bố không** — và cùng với nó là hai câu hỏi nữa, hỏi trước: bài toán thật sự là gì (đầu vào, đầu ra, tín hiệu giám sát), và họ giả định gì mà không nói ra.

Học xong chương này bạn làm được gì mà trước đó không làm được? Bạn đọc một paper lạ trong bốn mươi phút và phát biểu được ba câu: đóng góp thật nằm ở đâu, bằng chứng nào chưa đủ để kết luận, và thí nghiệm nào sẽ khiến bạn đổi ý. Đó là điều kiện đủ để ra quyết định đắt nhất mà một kỹ sư tự học phải ra thường xuyên — có nên bỏ hai tuần tái hiện một phương pháp hay không.

TRỰC GIÁC

Đọc một paper giống nhận một *pull request* mà tác giả vừa viết code, vừa viết test, vừa được quyền chọn công khai lần chạy CI nào. Không ai phải nói dối để bạn hiểu sai: chỉ cần những lần chạy đỏ không nằm trong diff là đủ. Việc của người đọc, giống việc của người review, là đoán ra *hình dạng của những lần chạy không được đưa vào* — và biết cần yêu cầu thêm test nào thì mới đủ tin.

1.2 Ba lượt đọc và thứ tự bên trong một lượt (Three-Pass Reading)

A. Ba lượt và điểm dừng của mỗi lượt

Khung ba lượt của Keshav là điểm khởi đầu tốt vì nó cho phép bỏ dở một cách có kỷ luật [1]. Lượt một mất năm đến mười phút — tiêu đề, abstract, hình, kết luận — và kết thúc bằng đúng một quyết định nhị phân: rút hay đọc tiếp. Lượt hai mất khoảng một giờ, đọc phương pháp và thiết lập thí nghiệm nhưng bỏ mọi chứng minh, kết thúc bằng khả năng tóm tắt paper cho người khác kèm những chỗ mình chưa nắm. Lượt ba là đọc để tái dựng: tự đặt mình vào vị trí tác giả

và dừng lại từng bước, mỗi chỗ mình chọn khác đi là một chỗ đáng ghi. Chỉ vài paper mỗi năm xứng đáng với lượt ba.

Bảng 1.1. Ba lượt đọc: mỗi lượt có một sản phẩm khác nhau, và điều kiện dừng của lượt trước quyết định có bước vào lượt sau hay không.

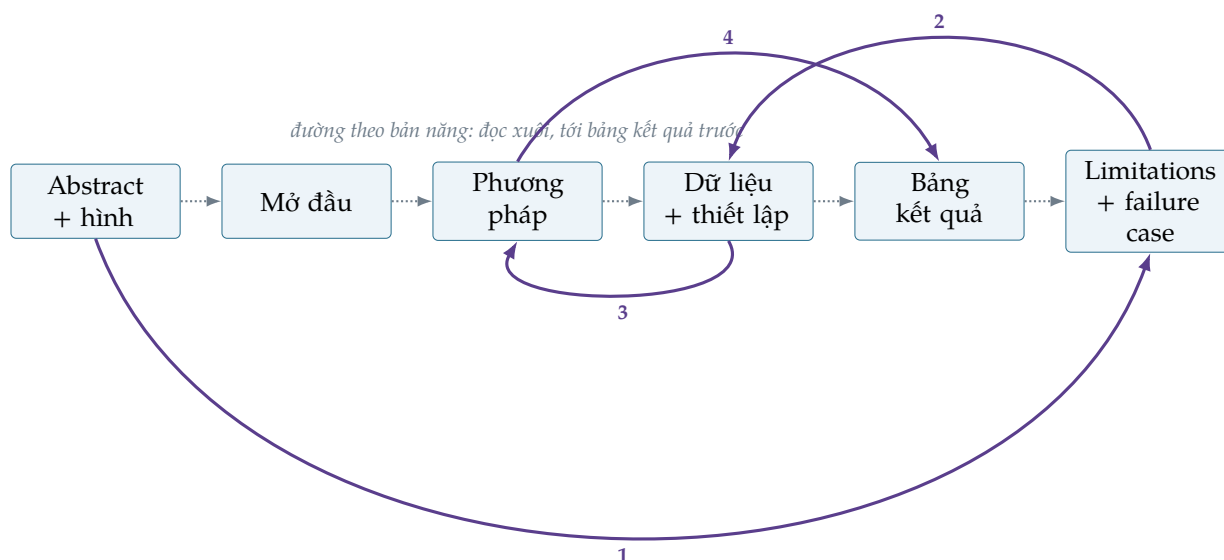
Lượt	Chi phí	Sản phẩm	Dừng lại nếu
1 — định vị	5–10 phút	Một quyết định nhị phân: vứt hay đọc tiếp	Bài toán không phải bài toán của bạn, hoặc giả định gốc đã sai với bối cảnh của bạn
2 — hiểu	≈ 1 giờ	Tóm tắt được cho người khác, kèm danh sách chỗ chưa nắm	Bảng chứng không tương xứng với tuyên bố; ghi lại lý do rồi dừng
3 — tái dựng	Nửa ngày trở lên	Dựng lại được thiết kế và biết mình sẽ chọn khác ở đâu	Chỉ dành cho paper bạn định cài lại hoặc xây tiếp lên

B. Thứ tự đọc bên trong một lượt

Điều mà khung này không nói, và là chỗ dễ sai nhất trong thị giác máy tính, là **thứ tự đọc bên trong một lượt**. Bản năng dẫn ta tới bảng kết quả trước, vì đó là chỗ có con số. Nên làm ngược lại, theo đúng trình tự sau. Trước hết đọc phần *limitations* và các trường hợp hỏng — thường nằm cuối, thường ngắn, thường là phần được đầu tư ít công sức nhất, và chính vì thế nó rõ ràng nhiều sự thật nhất. Nó cho bạn biết tác giả *tự biết* mình yếu ở đâu, và nếu bạn đọc nó trước thì mọi con số sau đó được đọc trong đúng bối cảnh của chúng. Nếu một paper không có phần này, đó là tín hiệu cảnh báo chứ không phải dấu hiệu hệ thống hoàn hảo. Tiếp theo mới là phần dữ liệu: dữ liệu nào, chia thế nào, nhân từ đâu, có trùng lặp giữa train và test không. Cách chia dữ liệu quyết định con số có nghĩa hay không, và một bảng kết quả đọc trước phần dữ liệu là một bảng kết quả chưa được diễn giải. Bảng kết quả đọc sau cùng, và khi đó nó chỉ còn là xác nhận hay bác bỏ những gì bạn đã dự đoán từ hai phần trước.

↔ **tiền đề** B/07 — một con số chỉ mang nghĩa bên trong giao thức chia dữ liệu đã sinh ra nó; đọc bảng trước phần dữ liệu là đọc một đại lượng chưa được định nghĩa

Hình 1.1 đặt cạnh nhau hai đường đi qua cùng một paper: đường theo bản năng và đường nên đi.



Hình 1.1. Cùng một paper, hai đường đi. Đường chấm mờ là thứ tự xuất bản; đường đậm đánh số là thứ tự nên đọc: limitations trước, rồi phần dữ liệu, rồi phương pháp, và bảng kết quả sau cùng. Thứ thay đổi không phải lượng thông tin nhận được mà là *bối cảnh* mà mỗi con số được đọc trong đó.

C. Đọc code cùng paper

Cuối cùng, trong thị giác máy tính, đọc paper mà không mở code là đọc một nửa. Các chi tiết thật sự quyết định con số hầu như luôn nằm trong repo, và hầu như không bao giờ nằm đủ trong bài viết. Danh sách quen thuộc gồm **gán mẫu** (*label assignment*) cho các **anchor**, tập **tăng cường dữ liệu** (*data augmentation*), lịch **learning rate**, ngưỡng **triệt phi cực đại** (*non-maximum suppression, NMS*), và việc có dùng trung bình trượt của trọng số (*EMA*) hay không. Bằng chứng rõ nhất cho điều này đến từ chính một paper detection: khi tách bạch từng khác biệt giữa dòng anchor-based và anchor-free, phần lớn khoảng cách hoá ra nằm ở **định nghĩa mẫu dương/âm** chứ không ở kiến trúc đầu ra [2]. Một chi tiết vốn bị coi là tiểu tiết cài đặt lại chính là đóng góp.

↔ **trường hợp riêng của** C/14 — gán mẫu trong detector là ví dụ điển hình nhất của loại chi tiết quyết định kết quả mà chỉ code mới nói ra

1.3 Ghi chú và mục “chỗ tôi không tin” (Paper Notes)

Ghi chú tóm tắt lại nội dung paper gần như vô dụng: sáu tháng sau bạn sẽ đọc lại chính abstract nhanh hơn là đọc ghi chú của mình. Ghi chú có giá trị là ghi chú chứa những thứ *không có* trong paper. Một khuôn năm mục vừa đủ và đáng giữ nguyên qua nhiều năm:

1. Bài toán trong ba dòng: đầu vào, đầu ra, tín hiệu giám sát. Nếu viết không nổi ba dòng này, bạn chưa đọc xong.
2. Ý tưởng lõi trong một câu, không dùng tên riêng của phương pháp.
3. Bằng chứng mạnh nhất trong bài — cụ thể là thí nghiệm nào, không phải con số nào.
4. **Chỗ tôi không tin** — và vì sao.
5. Thí nghiệm nào sẽ khiến tôi đổi ý.

Mục bốn là mục quan trọng nhất và cũng là mục duy nhất tạo ra một vòng phản hồi. Đọc mà không ghi lại sự hoài nghi thì sự hoài nghi tan biến, và bạn không bao giờ biết trực giác của mình đúng hay sai. Ghi lại thì sáu tháng sau, khi paper cải tiến hoặc paper phản biện xuất hiện, bạn đối chiếu được: lần này tôi nghi đúng chỗ, lần kia tôi nghi sai vì tôi không biết một ràng buộc thực tế. Đó là cách duy nhất để khả năng đọc paper của một người thật sự lên tay. Vòng này vận hành đúng như “dự đoán trước, đối chiếu sau” của thí nghiệm ở chương 24: cùng một cơ chế học, chỉ khác đối tượng.

Mục năm tồn tại để buộc sự hoài nghi phải cụ thể. “Tôi thấy nghi ngờ” là một cảm giác; “tôi sẽ tin nếu họ chạy phương pháp này với *baseline* được tune bằng cùng ngân sách trên ba *seed*” là một mệnh đề kiểm chứng được. Không viết nổi mục năm thì mục bốn chỉ là định kiến.

1.4 Đóng góp thật và cách kể chuyện (Contribution vs. Narrative)

Phần lớn cải tiến được công bố trong thị giác máy tính là tổ hợp của ba nguồn: một ý tưởng mới, một **công thức huấn luyện** (*training recipe*) tốt hơn, và nhiều dữ liệu hơn. Cách kể chuyện chuẩn của paper quy toàn bộ chênh lệch cho nguồn thứ nhất, vì hai nguồn còn lại không kể thành câu chuyện được. Đây không phải phỏng đoán mà là điều đã được đo nhiều lần. Dòng ConvNeXt lấy một ResNet rồi đưa dần từng thành phần của công thức huấn luyện và thiết kế hiện đại vào. Phần lớn khoảng cách so với Swin biến mất trước khi chạm tới bất cứ thứ gì mang bản chất *attention* [3]. Trong metric learning, một đánh giá lại có kiểm soát cho thấy đường cong tiến bộ gần như phẳng suốt nhiều năm khi mọi phương án được **tinh chỉnh siêu tham số** (*hyperparameter tuning*) bằng cùng ngân sách và đo bằng cùng giao thức [4]. Trong hệ khuyến nghị, kết quả tương tự: phần lớn phương pháp neural được công bố không vượt nổi baseline cổ điển được tune tử tế [5]. Trong học tăng cường, phương sai giữa các seed lớn tới mức nhiều so sánh đã công bố không phân biệt được với nhiễu [6].

Bài học rút ra không phải là hoài nghi mọi thứ, mà là một phép thử duy nhất áp cho mọi paper: **nếu bỏ đóng góp chính đi nhưng giữ nguyên toàn bộ công thức huấn luyện, con số còn lại bao nhiêu — và paper có cho bạn biết điều đó không?** Paper trả lời được câu này (bằng một *ablation* có kiểm soát) đáng tin hơn hẳn paper không trả lời, độc lập với việc con số cuối cùng đẹp tới đâu. Cũng nên phân biệt: một công thức huấn luyện tốt hơn *là* một đóng góp thật, chỉ là một đóng góp khác loại — nó chuyển giao được sang mọi kiến trúc, trong khi một module mới thường không. Vấn đề chỉ nằm ở việc gọi tên sai.

↔ **cùng nguyên lý với C/12** — cơ chế huấn luyện giải thích phần lớn chênh lệch giữa hai hệ thống — cùng một quan sát, ở đây nhìn từ phía người đọc paper

1.5 Nhận ra một so sánh không công bằng (Unfair Comparison)

Phần lớn so sánh lệch không đến từ ý đồ xấu. Nó đến từ bất đối xứng công sức: tác giả dành sáu tháng cho phương pháp của mình và một buổi chiều cho baseline. Cơ chế toán học phía sau đơn giản và đã được mô tả rõ trong bối cảnh tìm siêu tham số: hiệu năng tốt nhất quan sát được là một hàm tăng theo số lần thử, nên hai phương án được thử với ngân sách khác nhau thì không so được, kể cả khi mọi thứ khác giống hệt [7]. Bảng 1.2 liệt kê các dấu hiệu thường gặp cùng cách kiểm tra nhanh và — quan trọng không kém — trường hợp mà dấu hiệu đó vô hại.

Bảng 1.2. Các dạng so sánh không công bằng hay gặp trong paper thì giác máy tính. Cột cuối tồn tại để bạn không biến sự hoài nghi thành phần xạ bác bỏ mọi thứ.

Dấu hiệu	Cơ chế gây lệch	Kiểm tra nhanh	Khi nào vô hại
Số baseline chép từ paper cũ	Khác thể hệ recipe: epoch, augmentation, độ phân giải	Đối chiếu số epoch và tập augmentation của cả hai	Baseline được chạy lại trong cùng codebase và ghi rõ điều đó
So ở cùng số epoch	Phương pháp mới đắt hơn mỗi bước nên được nhiều tính toán hơn	Tìm chi phí mỗi bước hoặc tổng GPU-giờ	Chi phí bước gần bằng nhau và có báo cáo
Không ghi số seed	Chênh lệch có thể nằm dưới sàn nhiễu	Tìm dấu $\pm$ và số lần chạy trong chú thích bảng	Benchmark có phương sai rất nhỏ và điều đó được chứng minh
Ablation chạy trên tập test	Cấu hình được chọn trên chính tập dùng để báo cáo	Đọc chú thích bảng ablation, không đọc bảng chính	Ghi rõ chọn trên val, báo cáo trên test một lần
Ngân sách tune bất đối xứng	Tối ưu nhiều lần cho mình, ít lần cho đối thủ	Tìm số cấu hình đã thử cho mỗi phương án	Cả hai dùng chung không gian tìm kiếm và cùng số lần thử

CẠM BÃY

Cạm bẫy tinh vi nhất không nằm ở bảng kết quả mà ở việc *đọc bảng trước phần dữ liệu*. Khi đã nhìn thấy con số, bạn sẽ vô thức đi tìm lý do để tin nó, và phần dữ liệu đọc sau đó trở thành thủ tục. Đảo thứ tự lại thì cùng một bảng đó bỗng đọc ra khác hẳn — đây là hiệu ứng neo, không phải vấn đề kiến thức, nên biết về nó không đủ để tránh; phải đổi thứ tự thao tác.

1.6 Theo dõi một lĩnh vực mà không bị ngập (Field Tracking)

Đọc theo dòng thời gian là cách chắc chắn để bị ngập, vì luồng paper mới không có cấu trúc. Cách bền hơn là **đọc theo cụm**: một paper gốc, các paper cải tiến trực tiếp lên nó, và ít nhất một paper phản biện hoặc đánh giá lại. Ba loại bổ khuyết cho nhau — paper gốc cho ý tưởng ở dạng thuần khiết nhất, paper cải tiến cho biết chỗ nào của ý tưởng đó thật sự chịu lực, paper phản biện cho biết phần nào của con số đến từ chỗ khác. Thiếu loại thứ ba là lý do phổ biến nhất khiến một người nắm rất nhiều tên phương pháp mà không có phán đoán nào.

Sản phẩm nên có của việc theo dõi là một **cây phả hệ** (*genealogy of a field*), không phải một danh sách. Mỗi node đúng một dòng, và dòng đó phải có dạng “giải quyết vấn đề X mà node cha không giải được”. Ràng buộc một dòng ấy chính là bộ lọc: nếu bạn không viết nổi câu đó cho một paper, thì hoặc bạn chưa hiểu nó, hoặc nó chưa cần thiết với bạn — và cả hai kết luận đều hữu ích. Mười lăm node cho một mảng hẹp thường đủ để bạn định vị bất kỳ paper mới nào trong vài phút, vì paper mới gần như luôn là con của một node đã có.

Cuối cùng, nên duy trì *hai* danh sách paper nền chứ không phải một:

- **Nhánh nguyên lý mô hình** — kiến trúc, cơ chế huấn luyện, các dòng phương pháp. Đây là nhánh ai cũng có, và cũng là nhánh bán rã nhanh nhất: một nửa số tên trong đó sẽ hết quan trọng sau vài năm.
- **Nhánh dữ liệu, đo lường và đánh giá** — cách xây benchmark, lỗi nhãn, dịch chuyển phân phối, giao thức so sánh. Nhánh này gần như luôn bị bỏ qua, bán rã chậm hơn nhiều, và quyết định phần lớn khả năng phán đoán. Với một kỹ sư đã quen đo đạc hệ thống thật, đây là chỗ lợi thế sẵn có lớn nhất — bạn đã có bản năng đúng, chỉ thiếu tên gọi và tài liệu tham chiếu.

↔ **hệ quả của E/24** — mọi câu hỏi bạn dùng để nghi ngờ bảng của người khác đều bắt nguồn từ cách bằng chứng được sinh ra ở chương 24

1.7 Giới hạn và trường hợp hỏng

Quy trình trên giả định ba điều: paper có phần ablation chạy nghiêm túc, có code công khai, và lĩnh vực đã đủ chín để có paper phản biện. Cả ba giả định đều sai trong hai năm đầu của một hướng mới, và đó chính là lúc bạn cần phán đoán nhất. Trong giai đoạn ấy, thứ thay thế được là đọc chéo giữa các nhóm độc lập: nếu ba nhóm không liên quan báo cáo cùng một hiện tượng với ba thiết lập khác nhau, hiện tượng đó đáng tin hơn một paper duy nhất có bảng đẹp.

Chế độ hỏng thứ hai nằm ở phía người đọc: hoài nghi có chi phí. Áp dụng đủ mạnh, bạn sẽ không tin gì cả và không quyết định được gì — trạng thái này tệ ngang với cả tin. Sự hoài nghi hữu ích luôn kèm một mệnh đề cụ thể có thể kiểm chứng, tức là mục nằm trong khuôn ghi chú ở trên; sự hoài nghi không kèm mệnh đề đó chỉ là sự trì hoãn được nguy trang. Chế độ hỏng thứ ba tinh vi hơn: một cây phả hệ đẹp tạo cảm giác đã hiểu lĩnh vực, trong khi thứ bạn có mới chỉ là bản đồ tên gọi. Phép thử là thử dự đoán kết quả của một thí nghiệm chưa đọc — nếu không dự đoán nổi thì bản đồ vẫn còn rỗng.

CẦN NHỚ

Nếu chỉ mang theo một câu: *bỏ đóng góp chính đi nhưng giữ nguyên công thức huấn luyện thì còn lại bao nhiêu?* Câu hỏi đó tách đóng góp thật khỏi cách kể chuyện nhanh hơn mọi thứ khác, và nó áp dụng cho cả paper của người khác lẫn công việc của chính bạn. [T2]

1.8 Thực hành

Công cụ và tài nguyên. [alphaXiv](#) và [arxiv-sanity](#) để lọc luồng paper mới; Semantic Scholar và Connected Papers để dựng cây trích dẫn; [huggingface.co/papers](#) cho thảo luận hằng ngày. Các bảng xếp hạng kiểu [paperswithcode](#) nay đã phân tán, nên bám **benchmark** chính thức của từng mảng (COCO, ADE20K, RF100-VL, WILDS, VBench) thay vì một bảng tổng. Danh sách công cụ này chốt tại tháng 9/2026 và sẽ cũ trong sáu đến mười hai tháng; riêng thói quen dựng cây phả hệ và đọc theo cụm thì không.

Câu hỏi kiểm tra hiểu. Trong paper bạn vừa đọc: nếu bỏ đóng góp chính nhưng giữ nguyên recipe huấn luyện thì còn lại bao nhiêu, và paper có cho bạn biết điều đó không? Baseline của họ được tune bằng bao nhiêu ngân sách so với phương pháp mới? Phần dữ liệu có cho phép bạn tự dựng lại cách chia train/val/test không?

Mini project. Dựng cây phả hệ mười lăm node cho một mảng hẹp bạn quan tâm, mỗi node đúng một dòng theo mẫu “giải quyết vấn đề X mà node cha không giải được”. Node nào viết không nổi thì bỏ ra khỏi cây và ghi lại vì sao.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
●●● Ba lượt đọc	Khung đọc có ba mức đầu tư: 5–10 phút để quyết định có đọc tiếp, khoảng một giờ để tóm tắt được cho người khác, và nửa ngày trở lên để dựng lại thiết kế. Lượt ba chỉ dành cho paper bạn định cài lại.
●●● Limitations	Phần cuối bài, nơi tác giả tự nêu chỗ yếu. Nó được đầu tư ít công nhất nên rò rỉ nhiều sự thật nhất; đọc nó trước sẽ đặt bối cảnh cho mọi con số phía sau.
●● Vì sao đọc limitations trước bảng kết quả?	Vì thứ tự đọc quyết định cái neo. Đọc bảng trước thì bạn dành phần còn lại để đi tìm lý do tin nó; đọc limitations trước thì cùng những con số ấy được đọc trong bối cảnh tác giả đã tự thừa nhận.
●● Gán mẫu (label assignment)	Quy tắc quyết định anchor hay điểm nào là mẫu dương, mẫu nào là âm, khi huấn luyện detector. Nó hầu như chỉ có trong code, và có lần nó chính là toàn bộ khoảng cách giữa hai dòng phương pháp.
●● Vì sao phải mở code cùng paper?	Vì trong thị giác máy tính, các chi tiết quyết định con số nằm trong repo chứ không trong bài viết: gán mẫu, augmentation, lịch learning rate, ngưỡng NMS, có dùng EMA hay không.
●●● Công thức huấn luyện	Toàn bộ lựa chọn quanh việc huấn luyện mà không phải kiến trúc: số epoch, lịch learning rate, augmentation, khởi tạo, EMA. Đối riêng nó đã đủ tạo ra phần lớn chênh lệch giữa hai hệ thống.
●●● “Chỗ tôi không tin”	Mục thứ tư trong ghi chú năm mục, ghi lại sự hoài nghi kèm thí nghiệm sẽ khiến bạn đổi ý. Nó là thứ duy nhất kiểm chứng được về sau, nên là vòng phản hồi duy nhất trong việc đọc paper.
●●● Ngân sách tinh chỉnh	Số cấu hình đã thử cho mỗi phương án. Kết quả tốt nhất quan sát được là cực đại của một mẫu, nên hai phương án thử với ngân sách khác nhau thì không so được, kể cả khi mọi thứ khác giống hệt.
●● Vì sao “baseline yếu” là lỗi phổ biến nhất?	Vì nó rẻ: tác giả dành sáu tháng cho phương pháp của mình và một buổi chiều cho baseline. Bất đối xứng công sức đó tạo ra chênh lệch mà không cần ai gian dối.
●● Ablation trên tập test	Chọn cấu hình trên chính tập dùng để báo cáo, tức là một dạng rò rỉ. Tìm dấu hiệu này ở chú thích bảng ablation, không phải ở bảng chính.
●●● Paper báo cải tiến 1,0 mAP nhưng không ghi số seed — đọc thế nào?	Chưa kết luận được gì: chênh lệch có thể nằm dưới sàn nhiễu giữa các seed. Tìm dấu ± và số lần chạy; không có thì hạ mức tin cậy chứ đừng suy đoán bù.
●● Paper không có phần limitations — suy ra gì?	Đó là tín hiệu cảnh báo, không phải dấu hiệu hệ thống hoàn hảo. Mọi hệ thống đều trả giá ở đâu đó; không tìm ra nghĩa là bạn chưa hiểu nó, hoặc tác giả không muốn bạn hiểu.
●●● Bạn tái hiện một repo và thấp hơn số công bố 2 điểm — kiểm tra gì trước?	Theo thứ tự: phiên bản dữ liệu và tiền xử lý, rồi công thức huấn luyện (epoch, augmentation, độ phân giải), rồi phần cứng và cách tính metric. Nghi phương pháp là bước cuối, không phải bước đầu.
●● Vì sao công thức huấn luyện vẫn là đóng góp thật?	Vì nó chuyển giao được sang mọi kiến trúc, trong khi một module mới thường không. Nó chỉ là đóng góp <i>khác loại</i> ; vấn đề nằm ở việc gọi tên sai, không ở giá trị.
●● Đọc theo cụm	Đọc một paper gốc, các paper cải tiến trực tiếp lên nó, và ít nhất một paper phản biện. Thiếu loại thứ ba là lý do phổ biến nhất khiến một người nhớ rất nhiều tên mà không có phán đoán nào.
● Cây phả hệ	Danh sách node, mỗi node đúng một dòng: “giải quyết vấn đề X mà node cha không giải được”. Ràng buộc một dòng ấy chính là bộ lọc; viết không nổi thì node đó chưa cần với bạn.
● Hai danh sách nền	Một nhánh về nguyên lý mô hình, một nhánh về dữ liệu và đánh giá. Nhánh thứ hai bán rã chậm hơn nhiều, hay bị bỏ qua, và là lợi thế sẵn có của người làm hệ thống.

● ● ● phải thuộc ● ● nên nắm ● biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
Quy trình bình duyệt CVPR / NeurIPS	Biểu mẫu review yêu cầu đúng bộ câu hỏi của chương: điểm mạnh, điểm yếu, thí nghiệm còn thiếu, câu hỏi cho tác giả	Đây là nơi kỹ năng này được hành nghề hằng ngày; mẫu review công khai
Reproducibility checklist (NeurIPS, ICLR)	Ép khai báo số seed, độ lệch chuẩn, ngân sách tính toán — chính bốn dấu hiệu ở bảng so sánh lệch	Thành thông lệ từ khoảng 2019, nay gần như bắt buộc
Semantic Scholar, Connected Papers	Dùng cây trích dẫn tự động, tức cây phá hệ ở dạng công cụ	Hạ tầng; bền hơn mọi tên mô hình
Dòng ConvNeXt [3]	Cả paper là một ablation recipe-so-với-kiến trúc có kiểm soát, đúng phép thử “bỏ đóng góp chính, giữ recipe”	Ví dụ mẫu mực để đọc trước khi tự viết ablation
Đánh giá lại metric learning [4]	Hiện thực hoá phép thử “cùng ngân sách tính chỉnh, cùng giao thức” trên cả một mảng	Kết quả đã công bố, không phải phỏng đoán

MINI PROJECT SLAM · Đọc ba lượt một paper frontend học được, và viết ghi chú có mục “chỗ tôi không tin” · nửa ngày

Mục tiêu	biến quy trình ba lượt của chương thành một thói quen đo được. Đối tượng là đúng loại paper bạn phải quyết định có tái hiện hay không: một paper về frontend học được cho visual SLAM, tức đặc trưng học được, so khớp học được, hoặc odometry học được.
Dữ liệu	một paper bạn chưa đọc, kèm repo của nó nếu có. Chọn paper báo cáo kết quả trên EuRoC hoặc TUM-VI để bạn đối chiếu được với số bạn đã tự đo.
Các bước	lượt một năm phút (abstract, hình, kết luận) rồi quyết vớt hay đọc tiếp; lượt hai theo đúng thứ tự limitations → dữ liệu → phương pháp → bảng kết quả; mở repo tìm ba chi tiết không có trong bài viết (ngưỡng so khớp, augmentation, lịch huấn luyện); viết ghi chú năm mục theo khuôn ở mục “Ghi chú”.
Xong khi	bạn nêu được một chi tiết cụ thể trong phần thí nghiệm mà nếu đổi đi thì kết luận của paper có thể sụp. Ví dụ: chuỗi EuRoC nào bị loại khỏi bảng, hay baseline chạy ở độ phân giải nào. Và bạn viết được mục “thí nghiệm sẽ khiến tôi đổi ý” dưới dạng một cấu hình chạy được.
Nối với	E / 24 cho cách kiểm tra chính chi tiết đó bằng số; B / 07 cho phần giao thức đánh giá mà bạn dùng để đọc phân dữ liệu.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- Vì thứ tự đọc quyết định cái neo. Đọc limitations trước thì mọi con số sau đó được diễn giải trong bối cảnh tác giả tự thừa nhận; đọc bảng trước thì bạn dành phần còn lại để đi tìm lý do tin nó. Cùng lượng thông tin, khác kết luận.
- Số seed kèm độ lệch chuẩn, ngân sách tính chỉnh của từng phương án, và việc ablation chạy trên val hay test — xem Bảng 1.2. Chúng không nằm trong bảng kết quả vì bảng kết quả được thiết kế để thuyết phục, còn ba chi tiết đó chỉ làm yếu đi câu chuyện.
- Phần lớn là ngân sách tính chỉnh và công thức huấn luyện, tức cỡ mẫu của một cuộc tìm kiếm, chứ không phải ý tưởng. Điều đó không có nghĩa recipe vô giá trị — nó là đóng góp thật, chỉ khác loại và chuyển giao được rộng hơn một module mới.
- Mục “chỗ tôi không tin” cộng mục “thí nghiệm sẽ khiến tôi đổi ý”. Chúng kiểm chứng được về sau, nên tạo ra vòng phản hồi duy nhất trong việc đọc paper: sáu tháng sau bạn biết mình đã nghi đúng chỗ hay nghi sai vì thiếu một ràng buộc thực tế.

NÓI LẠI THẬT ĐƠN GIẢN

Bạn đang tìm mua nhà, và mỗi ngày có ba mươi tin rao mới. Mỗi tin đều do người bán viết: ảnh chụp góc rộng lúc trời nắng, phòng nào chật thì không có ảnh, và không tin nào nhắc tới cái cống trước cửa. Không ai nói dối cả; họ chỉ được quyền chọn tấm ảnh nào đăng lên. Khó khăn của bạn không phải là đọc không hiểu — tin nào bạn cũng hiểu — mà là không đủ thời gian đi xem hết, nên phải đoán từ trang giấy xem căn nào đáng đi.

Mẹo gồm ba việc. Một, đảo thứ tự đọc: xem phần người bán tự nhận là điểm yếu trước, rồi tới giấy tờ và diện tích thật, và để giá cùng ảnh đẹp xuống sau cùng. Hai, hỏi xem họ đã sửa sang bao nhiêu trước khi chụp, vì một căn vừa sơn lại toàn bộ và một căn để nguyên thì không đặt cạnh nhau được. Ba, ghi lại đúng một dòng “chỗ tôi thấy nghi” cho mỗi tin, để sáu tháng sau quay lại bạn biết lần nào mình nghi đúng và lần nào nghi sai.

Cái giá phải trả: đọc một tin lâu hơn hẳn, và nếu quen nghi ngờ quá tay thì bạn không bao giờ dám đi xem căn nào. Nghi ngờ chỉ có ích khi kèm một câu hỏi cụ thể bạn định hỏi người bán.

Một câu: Người viết được quyền chọn ảnh nào đăng lên, nên việc của bạn là đoán ra những tấm không được đăng trông như thế nào.

1.9 Kết nối

Chương này là mặt đọc của cùng một kỹ năng mà [E/24-thi-nghiem-va-ablation](#) dạy ở mặt viết: mọi tiêu chí bạn dùng để nghi ngờ ablation của người khác đều là tiêu chí bạn phải tự thoả khi làm ablation của mình, nên hai chương nên đọc gần nhau. [B/07-chia-du-lieu-va-danh-gia](#) là tiền đề: không đọc được phần dữ liệu thì không đọc được bảng kết quả. [B/03-giai-phau-he-thong](#) cho bạn khung ba tầng để phân loại một đóng góp là ý tưởng hay chi tiết cài đặt. [D/22-tai-lap-va-mlops](#) là mặt đối lập hữu ích: nó cho thấy vì sao ngay cả tác giả trung thực cũng không tái lập được số của chính mình sau sáu tháng, nên khoảng cách giữa paper và thực tế không phải lúc nào cũng là dấu hiệu của sự thiếu trung thực.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao đọc phần limitations trước bảng kết quả lại đổi được cách bạn diễn giải cùng một con số, dù thông tin nhận được là như nhau?
2. Một paper báo cáo cải tiến 1,0 mAP nhưng không ghi số seed: ba cách giải thích khác nhau cho con số đó là gì, và với mỗi cách thì chi tiết nào trong paper sẽ giúp bạn loại trừ nó?
3. Vì sao “công thức huấn luyện tốt hơn” là một đóng góp thật nhưng lại là loại đóng góp khác với “một module mới”? Sự khác nhau nằm ở đâu về mặt khả năng chuyển giao?
4. Mục “chỗ tôi không tin” vô dụng nếu thiếu mục nào đi kèm, và vì sao?
5. Khi nào việc đọc theo cụm hổng, và bạn thay bằng gì trong giai đoạn đó?
6. Cây phả hệ có thể tạo ảo giác hiểu biết. Phép thử nào phát hiện được ảo giác đó?

Nguồn

- [1] S. Keshav, *How to Read a Paper*, ACM SIGCOMM Computer Communication Review, 2007.
- [2] S. Zhang, C. Chi, Y. Yao, et al., “Bridging the Gap Between Anchor-Based and Anchor-Free Detection via Adaptive Training Sample Selection,” in *CVPR*, 2020.
- [3] Z. Liu, H. Mao, C.-Y. Wu, et al., “A ConvNet for the 2020s,” in *CVPR*, 2022.
- [4] K. Musgrave, S. Belongie, and S.-N. Lim, “A Metric Learning Reality Check,” in *ECCV*, 2020.
- [5] M. Ferrari Dacrema, P. Cremonesi, and D. Jannach, “Are We Really Making Much Progress? A Worrying Analysis of Recent Neural Recommendation Approaches,” in *RecSys*, 2019.
- [6] P. Henderson, R. Islam, P. Bachman, et al., “Deep Reinforcement Learning That Matters,” *AAAI*, 2018.
- [7] J. Bergstra and Y. Bengio, “Random Search for Hyper-Parameter Optimization,” in *JMLR*, 2012.
- [8] P. Goyal, P. Dollár, R. Girshick, et al., “Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour,” *arXiv:1706.02677*, 2017.
- [9] C. Zhang, S. Bengio, M. Hardt, et al., “Understanding deep learning requires rethinking generalization,” in *ICLR*, 2017.
- [10] J. Demšar, “Statistical Comparisons of Classifiers over Multiple Data Sets,” *JMLR*, vol. 7, pp. 1–30, 2006.
- [11] B. Recht, R. Roelofs, L. Schmidt, et al., “Do ImageNet Classifiers Generalize to ImageNet?” In *ICML*, 2019.
- [12] C. G. Northcutt, A. Athalye, and J. Mueller, “Pervasive Label Errors in Test Sets Destabilize Machine Learning Benchmarks,” in *NeurIPS Datasets and Benchmarks*, 2021.
- [13] D. Bolya, S. Foley, J. Hays, et al., “TIDE: A General Toolbox for Identifying Object Detection Errors,” in *ECCV*, 2020.
- [14] D. Hoiem, Y. Chodpathumwan, and Q. Dai, “Diagnosing Error in Object Detectors,” in *ECCV*, 2012.
- [15] L. van der Maaten and G. Hinton, “Visualizing Data Using t-SNE,” *JMLR*, vol. 9, pp. 2579–2605, 2008.
- [16] L. McInnes, J. Healy, and J. Melville, “UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction,” *arXiv:1802.03426*, 2018.
- [17] R. R. Selvaraju, M. Cogswell, A. Das, et al., “Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization,” in *ICCV*, 2017.
- [18] J. Adebayo, J. Gilmer, M. Muelly, et al., “Sanity Checks for Saliency Maps,” in *NeurIPS*, 2018.
- [19] C. Guo, G. Pleiss, Y. Sun, et al., “On Calibration of Modern Neural Networks,” in *ICML*, 2017.
- [20] Y. Ovadia, E. Fertig, et al., “Can You Trust Your Model’s Uncertainty?” In *NeurIPS*, 2019.
- [21] Y. Gal and Z. Ghahramani, “Dropout as a Bayesian Approximation: Representing Model Uncertainty in Deep Learning,” in *ICML*, 2016.

- [22] B. Lakshminarayanan, A. Pritzel, and C. Blundell, "Simple and Scalable Predictive Uncertainty Estimation Using Deep Ensembles," in *NeurIPS*, 2017.
- [23] A. N. Angelopoulos and S. Bates, *Conformal Prediction: A Gentle Introduction*. Foundations and Trends in Machine Learning, 2023.
- [24] D. Hendrycks and T. Dietterich, "Benchmarking Neural Network Robustness to Common Corruptions and Perturbations," in *ICLR*, 2019.
- [25] D. Hendrycks and K. Gimpel, "A Baseline for Detecting Misclassified and Out-of-Distribution Examples in Neural Networks," in *ICLR*, 2017.
- [26] M. Mitchell, S. Wu, et al., "Model Cards for Model Reporting," in *FAT**, 2019.

Thiết kế thí nghiệm, ablation và phân tích lỗi (Experiment Design, Ablation, Error Analysis)

PHỤ THUỘC B/07-chia-du-lieu-va-danh-gia (chia dữ liệu, chọn metric — chương này giả định bạn đã có một giao thức đánh giá đúng); D/22-tai-lap-va-mlops (config, tracking, nguồn phi tắt định); E/23-doc-paper là mặt đọc của cùng kỹ năng này.

TÓM TẮT

Chương này là về cách *sinh ra bằng chứng*: thiết kế một so sánh có nghĩa, cắt bỏ thành phần đúng cách, biết bao nhiêu lần chạy là đủ, và đọc lỗi để quyết định đầu tư tiếp vào đâu. Đọc xong bạn tính được cỡ mẫu cần thiết từ sán nhiều, chọn được giữa leave-one-out và oracle experiment cho đúng câu hỏi mình đang hỏi, và viết được một bảng ablation mà người hoài nghi nhất cũng dùng được. Với nền SLAM sẵn có, phần bạn còn thiếu không phải kỹ thuật lặp lại nhiều lần mà là ba thứ đặc thù deep learning: phi tắt định không sửa được, bất đối xứng ngân sách tune, và rò rỉ tập test tích lũy qua nhiều tháng. Nếu chỉ nhớ một điều: đo sán nhiều *trước*, nói về cải tiến *sau* — và một kết luận âm ghi kèm điều kiện thí nghiệm vẫn là một kết quả.

KIẾN THỨC CẦN CÓ TRƯỚC

- **ablation** — thí nghiệm bỏ đi hoặc thay thế đúng *một* thành phần của hệ thống rồi đo lại, để biết thành phần đó đóng góp bao nhiêu.
- **baseline** — phương pháp đối chứng mà kết quả mới phải vượt qua; baseline chưa được tinh chỉnh từ tế làm mọi so sánh sau đó vô nghĩa.
- **seed và phương sai giữa các lần chạy** — seed là số khởi tạo bộ sinh ngẫu nhiên; chạy lại cùng cấu hình với seed khác cho kết quả khác, và độ tản mát của các kết quả đó là phương sai giữa các lần chạy — phần bạn đã quen từ harness SLAM, ở đây chỉ đổi nguồn gây ra nó.
- **độ lệch chuẩn σ** — thước đo độ tản mát của một tập số quanh trung bình của chúng. Trong chương này σ luôn là độ lệch chuẩn *giữa các seed* của cùng một cấu hình, tức là “sán nhiều”.
- **khoảng tin cậy** — một khoảng số kèm mức tin cậy (thường 95%), tính từ dữ liệu, sao cho nếu lặp lại toàn bộ quy trình nhiều lần thì khoảng ấy chứa giá trị thật trong đúng tỉ lệ đó. Nó trả lời “tôi biết con số này chính xác tới đâu”, không trả lời “giả thuyết của tôi đúng hay sai”.
- **p-value** — xác suất quan sát được chênh lệch ít nhất bằng chênh lệch đã thấy, *nếu* giả thuyết “không có khác biệt” là đúng. Nó *không* phải xác suất giả thuyết của bạn đúng, và một p-value nhỏ trên tập test rất lớn có thể ứng với một chênh lệch nhỏ tới mức vô nghĩa về mặt triển khai.
- **cỡ hiệu ứng (effect size)** — độ lớn của chênh lệch, đo bằng đơn vị của bài toán (0,8 mAP) hoặc chuẩn hoá theo σ . Đây mới là đại lượng quyết định “có đáng triển khai không”.
- **oracle experiment** — thay một khối trong hệ thống bằng đáp án đúng, tức **chân trị** (*ground truth*), để đo *trần* đóng góp của khối đó. Nếu khối hoàn hảo cũng chỉ nâng hệ thống 2% thì mọi công sức tối ưu nó đều bị chặn trên bởi 2%.
- **mAP** — metric tổng hợp thông dụng cho phát hiện vật thể; ở đây chỉ cần hiểu là “điểm số càng cao càng tốt”, tính theo điểm phần trăm.

CÂU HỎI MỞ ĐẦU

- Q1. Bạn thử hai mươi cấu hình rồi lấy cái tốt nhất và thấy nó hơn baseline 1,1 điểm. Vì sao con số đó có thể hoàn toàn là nhiễu — và bạn tính ra điều đó bằng phép tính nào, trong ba mươi giây?
- Q2. Quy tắc “chạy ba lần” mà bạn đã dùng cho harness SLAM là đúng, nhưng ba là bao nhiêu phần trăm của cái cần thiết? Với sàn nhiễu 0,6 điểm, ba lần chạy bỏ sót bao nhiêu phần số cải tiến có thật?
- Q3. Bạn bỏ một module và mAP giảm 1,5. Kể ra bốn cách giải thích khác ngoài “module đó quan trọng” — và cách nào trong số đó luôn dẫn kết luận về cùng một hướng?
- Q4. Nếu một detector hoàn hảo cũng chỉ nâng hệ thống tổng thể 2%, bạn còn lý do gì để tối ưu detector — và làm sao biết được con số 2% ấy *trước* khi bỏ ba tháng?
- Q5. Vì sao một kết luận NO-GO không ghi điều kiện thí nghiệm lại có hại hơn là không ghi gì cả?

2.1 Đặt vấn đề

Chương 7 nói về *giao thức đánh giá*: chia dữ liệu thế nào, đo bằng metric gì. Chương này nói về một việc khác hẳn — *sinh ra bằng chứng*: làm thế nào để một so sánh có nghĩa, và làm thế nào để biết nên đầu tư công sức tiếp theo vào đâu. Đây là kỹ năng phân biệt người làm nghiên cứu với người chạy script, và nó gần như không được dạy ở đâu cả; hầu hết người làm nghề học nó bằng cách tự trả giá.

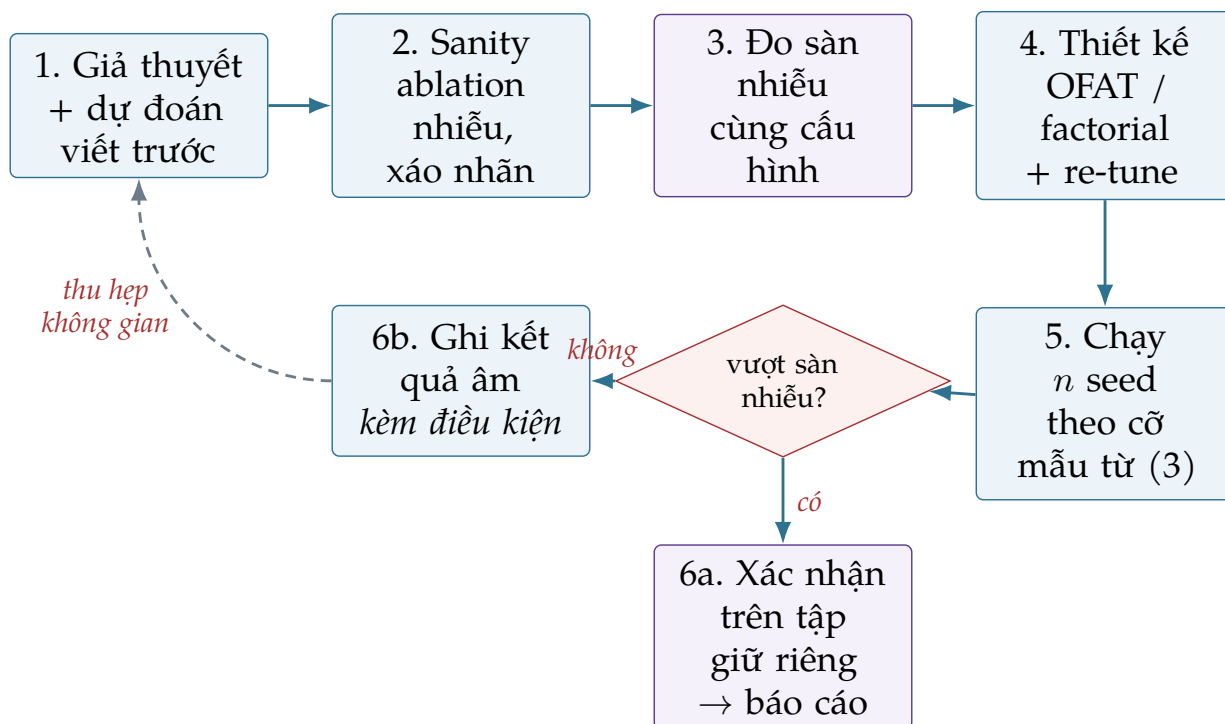
Bạn đã trả một phần cái giá đó rồi, ở phía SLAM. Khi dựng một harness chạy lặp trên EuRoC, bạn phát hiện chính hệ thống của mình là phi tất định: hệ thống có hai nguồn phi tất định cùng lúc.

- **Thứ tự duyệt container** — các container đánh chỉ mục bằng con trỏ (*pointer-ordered containers*) đổi thứ tự theo cách cấp phát bộ nhớ.
- **Tranh chấp luồng** (*race condition*) — giữa luồng tracking và luồng local mapping, khiến một cờ trạng thái đọc ra khác nhau giữa các lần chạy.

Hệ quả giống nhau ở cả hai nguồn: chênh lệch một inlier lan thành một quyết định keyframe khác, rồi thành một quỹ đạo khác. Từ đó bạn đặt quy tắc: mọi cấu hình chạy ít nhất ba lần với ba *seed* khác nhau. Bạn cũng đã tự viết những loạt ablation nhiều cấu hình và rút ra vài kết luận NO-GO, tức là đã biết bằng kinh nghiệm rằng phần lớn ý tưởng nghe hay đều chết, và rằng biết chúng chết là một kết quả.

Chương này vì thế không giảng lại nhóm C từ đầu. Nó hệ thống hoá những gì bạn đã biết thành một khung có tên gọi, để bạn áp được sang bài toán mới mà không phải phát hiện lại, và chỉ ra ba chỗ mà môi trường deep learning *khác*. Thứ nhất, nguồn phi tất định ở đây không sửa được — không có bug nào để tìm — nên mục tiêu phải chuyển từ “tái lập từng chữ số” sang “tái lập phân phối kết quả”. Thứ hai, không gian siêu tham số lớn hơn nhiều, nên bất đối xứng ngân sách tune giữa phương pháp mới và baseline trở thành nguồn kết luận sai lớn nhất, lớn hơn cả nhiễu seed. Thứ ba, tập test bị dùng lại hàng trăm lần trong nhiều tháng. Nó còn có thể đã nằm sẵn trong dữ liệu pretraining của mô hình nền bạn dùng. Đó là hai dạng rò rỉ mà một pipeline SLAM chạy trên EuRoC cũng mắc, chỉ là ít khi được gọi tên.

Hình 2.1 tóm cả chương thành một quy trình. Đọc nó trước rồi mới đọc năm nhóm bên dưới sẽ nhanh hơn nhiều, vì mỗi nhóm chỉ là một trạm trên đúng đường đi này.



Hình 2.1. Từ giả thuyết tới kết luận. Ba chỗ hay bị bỏ qua nhất được tô đậm hoặc rẽ nhánh. Bước 3: đo sản nhiều *trước* khi so sánh. Bước 5: tính cỡ mẫu từ chính sản nhiều đó. Bước 6b: coi nhánh “không vượt sản” là một kết quả phải ghi lại, không phải một lần chạy bỏ đi; mũi tên đứt là chỗ nó trả giá trị về cho vòng sau.

TRỰC GIÁC

Bỏ một module ra khỏi hệ thống rồi giữ nguyên learning rate cũ giống tháo giảm chấn khỏi hệ treo rồi vẫn chạy thử ở đúng tốc độ và đúng áp suất lốp cũ. Con số bạn đo được không phải “xe không có giảm chấn” mà là “xe không có giảm chấn *cộng* với một bộ tham số vận hành đã trở nên sai cho nó”. Sai lệch này luôn cùng chiều — nó thổi phồng tầm quan trọng của thành phần bị bỏ — nên nó không tự triệt tiêu khi bạn lặp lại nhiều lần.

2.2 Nhóm A — thiết kế so sánh (Comparison Design)

A. Ba loại câu hỏi, ba thiết kế

Lỗi phổ biến nhất trong thiết kế thí nghiệm không phải lỗi thống kê mà là lỗi phân loại: dùng một thiết kế để trả lời câu hỏi mà nó không trả lời được. Có đúng ba loại câu hỏi và chúng cần ba thiết kế khác nhau. “Cái này có hoạt động không” là so sánh với baseline, và thiết kế của nó đòi hỏi kiểm soát chặt mọi biến gây nhiễu cùng nhiều seed. “Vì sao nó hoạt động” là ablation, và thiết kế của nó đòi hỏi mỗi lần chỉ đổi một thứ, kèm re-tune. “Nó hoạt động tới đâu” là sensitivity và scaling. Thiết kế của nó cần nhiều điểm dọc một trục: kích thước dữ liệu, kích thước mô hình, hoặc mức nhiễu. Bù lại nó chấp nhận ít seed mỗi điểm, vì cái ta đọc là *xu hướng* chứ không phải một hiệu số. Lấy một bảng ablation ra để chứng minh phương pháp hoạt động, hoặc lấy một so sánh baseline ra để giải thích cơ chế, là hai cách trộn lẫn hay gặp nhất và cả hai đều cho kết luận không đứng vững.

B. Ba ràng buộc phải thoả trước khi con số có nghĩa

Trên nền đó, ba ràng buộc phải thoả trước khi bất kỳ con số nào có nghĩa. **Baseline mạnh là nghĩa vụ, không phải phép lịch sự:** so với baseline bị tune kém là cách tự lừa mình phổ biến nhất, và nó phổ biến vì rẻ. **Kiểm soát biến gây nhiễu:** cùng dữ liệu, cùng augmentation, cùng lịch huấn luyện, cùng số seed. Nếu phương pháp mới đắt hơn 1,4 lần mỗi bước mà vẫn chạy cùng số epoch, nó đã được tặng thêm 40% tính toán. Riêng lượng tính toán thêm đó đủ tạo khác biệt đo được với các công thức huấn luyện hiện đại [8]. **Ngân sách tune công bằng:** cơ chế ở đây thuần xác suất — giá trị tốt nhất quan sát được là cực đại của một mẫu, và kỳ vọng của cực đại tăng theo cỡ mẫu [7], nên một cải tiến 1% rút ra từ 50 lần thử so với baseline được thử 5 lần không phải là cải tiến mà là hiệu ứng của cỡ mẫu.

→ **hệ quả của E/23** — đây chính là dấu hiệu “ngân sách tính chính bất đối xứng” mà bạn dùng để đọc bảng kết quả của người khác

C. Một-biến-một-lần hay giai thừa

Còn một lựa chọn thiết kế nữa hay bị bỏ qua: một-biến-một-lần (OFAT) hay giai thừa (factorial). OFAT rẻ và trực giác, nhưng mù hoàn toàn với tương tác — nếu hai thành phần cộng hưởng hoặc triệt tiêu nhau, OFAT không thể phát hiện. Với ba thành phần, một thiết kế giai thừa đầy đủ có $2^3 = 8$ cấu hình, nhân ba seed là 24 lần chạy; OFAT chỉ cần cấu hình gốc cộng ba lần bỏ, tức 4 cấu hình và 12 lần chạy. Gấp đôi chi phí để đổi lấy toàn bộ tương tác bậc hai thường là món hời. Khi số thành phần lớn hơn, thiết kế giai thừa rút gọn cho phép trả ít hơn nhiều so với 2^k mà vẫn tách được các tương tác đáng kể.

2.3 Nhóm B — kỹ thuật ablation (Ablation Techniques)

A. Hai chiều bỏ thành phần, hai câu hỏi khác nhau

Có hai chiều bỏ thành phần, và chúng trả lời hai câu hỏi khác nhau. *leave-one-out* bỏ từng thành phần khỏi hệ thống đầy đủ; *add-one-in* thêm từng thành phần vào baseline trần. Khi các thành phần độc lập, hai chiều cho cùng con số. Khi có tương tác, chúng tách ra, và *chính khoảng cách giữa chúng là thông tin*. Ví dụ nhỏ nhất: giả sử thêm riêng A được +1,0, thêm riêng B được +1,0, nhưng A và B cùng lúc chỉ được +1,2. Khi đó *leave-one-out* báo “bỏ A mất 0,2” còn *add-one-in* báo “thêm A được 1,0”. Cả hai đều đúng; chúng chỉ trả lời hai câu khác nhau — “A còn đáng giữ trong hệ thống này không” và “A có tự nó tạo ra giá trị không”. Chỉ báo cáo một chiều rồi phát biểu như thể đó là “đóng góp của A” là chỗ mập mờ hay gập nhất trong các bảng ablation đã công bố.

B. Ba loại đối tượng, và luật re-tune

Ablation có ba loại theo đối tượng bị bỏ: kiến trúc, dữ liệu, và công thức huấn luyện. Hai loại sau gần như luôn bị bỏ qua, dù chúng gần như luôn giải thích phần lớn chênh lệch. Đây chính là mặt trong của kết luận ở chương 23 về đóng góp thật so với recipe: nếu bạn không ablate recipe, bạn không có cách nào biết đóng góp của mình có thật không.

Ràng buộc quan trọng nhất của cả nhóm là **ablation phải re-tune**, vì lý do ở hộp trực giác: bỏ một thành phần có tác dụng ổn định hoá thì learning rate tối ưu của phần còn lại thường nhỏ đi, nên giữ nguyên learning rate cũ vừa làm hệ thống tệ hơn vừa thổi phồng tầm quan trọng của thành phần vừa bỏ. Thoả hiệp thực dụng là re-tune *ít nhất* learning rate trên một lưới thô ba điểm và ghi rõ rằng chỉ có learning rate được re-tune.

C. Sanity ablation chạy trước

Trước mọi ablation tinh vi, hãy chạy nhóm **sanity ablation**: thay đầu vào bằng nhiễu, xáo trộn nhãn, cắt bỏ một modality, thay một module bằng hằng số. Nếu điểm không giảm thì hoặc có rò rỉ, hoặc thành phần đó vô dụng, và cả hai kết luận đều dừng cuộc điều tra lại tại chỗ. Nhóm này rẻ và có sức phá hoại giả thuyết cao nhất, nên chạy trước là đúng thứ tự kinh tế. Một lưu ý về diễn giải: mạng đủ lớn có thể học thuộc cả một tập nhãn ngẫu nhiên, nên “train loss vẫn giảm khi xáo nhãn” không phải dấu hiệu rò rỉ — chỉ có *điểm trên tập đánh giá* không giảm mới là [9].

D. Oracle experiment và ablation theo quy mô

Công cụ mạnh nhất để quyết định đầu tư lại là loại ablation ngược: **oracle** hay **skyline**, tức thay một khối bằng chân trị và đo trần đóng góp của khối đó. Nếu một detector hoàn hảo chỉ nâng hệ thống tổng thể lên 2%, thì mọi công sức tối ưu detector đều bị chặn trên bởi 2% và bạn nên đi chỗ khác. Bạn đã dùng đúng công cụ này ở phía SLAM khi thay tầng phát hiện vòng lặp bằng một oracle lấy từ chân trị để tách “không phát hiện được vòng lặp” khỏi “phát hiện được nhưng không dùng được” — và kết luận thu được đã định hướng lại toàn bộ hướng làm việc, đó là điều một ablation thông thường không làm được. Cuối cùng, mọi kết luận ablation đều gắn với quy mô: kết luận rút ra trên 1% dữ liệu thường đảo ngược ở 100%, vì các thành phần có tác dụng chống quá khớp mất ý nghĩa khi dữ liệu đủ nhiều. Luôn ghi ablation chạy ở quy mô nào.

→ **trường hợp riêng của B/03** — oracle experiment là cách trả lời bằng số cho câu hỏi “độ khó đã bị đẩy đi đâu”: nó xoá hẳn độ khó của một khối rồi đo phần còn lại

Bảng 2.1. Năm kiểu ablation, mỗi kiểu trả lời một câu hỏi khác nhau. Cột cuối là cột đáng giá nhất: nó cho biết khi nào con số thu được không có nghĩa như ta tưởng.

Kiểu	Trả lời câu hỏi	Giả định	Hỏng khi nào
Leave-one-out	Thành phần này còn đáng giữ không	Phần còn lại vẫn ở điểm vận hành tốt	Không re-tune; hoặc thành phần bị bỏ kéo theo thay đổi ngầm
Add-one-in	Thành phần này tự nó có giá trị không	Baseline trần vẫn huấn luyện ổn định	Có tương tác mạnh: con số không cộng lại thành hệ thống đầy đủ
Sanity	Có rò rỉ, hay thành phần vô dụng	Metric đánh giá nhạy với thứ bị phá	Đọc nhầm train loss thay vì điểm đánh giá
Oracle / skyline	Trần đóng góp của khối này là bao nhiêu	Chân trị cho khối đó tồn tại và đúng định dạng	Oracle làm đổi phân phối đầu vào của khối sau, cho trần lạc quan giả
Theo quy mô	Kết luận có bền theo lượng dữ liệu không	Xu hướng đơn điệu giữa hai đầu	Chỉ đo hai điểm; giao nhau nằm giữa mà không thấy

I 2.4 Nhóm C — thống kê tối thiểu (Minimal Statistics)

A. Sàn nhiễu trước, cỡ mẫu sau

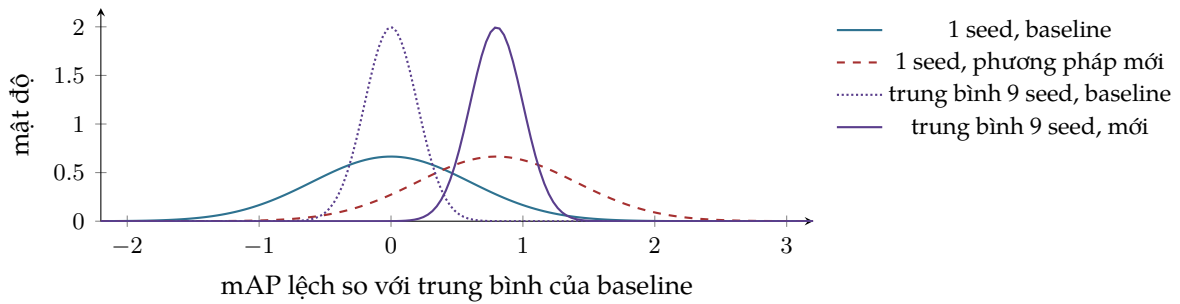
Quy tắc ba lần chạy của bạn đúng về nguyên tắc nhưng ba là *sàn*, không phải mục tiêu; số lần chạy cần thiết phụ thuộc vào tỉ số giữa hiệu ứng muốn phát hiện và độ lệch chuẩn giữa các seed. Thứ tự đúng vì thế là **đo sàn nhiễu (noise floor) trước, nói về cải tiến sau**: chạy *cùng một cấu*

hình nhiều lần chỉ đổi seed, lấy độ lệch chuẩn, và chỉ từ đó mới biết chênh lệch bao nhiêu là đáng nói. Một tổ chức không biết sản phẩm của mình sẽ dành nhiều tháng để đuổi theo nhiều.

Ví dụ tính tay được, dùng đúng những con số hay gặp trong detection. Giả sử độ lệch chuẩn giữa các seed là $\sigma = 0,6$ mAP và bạn muốn phát hiện một cải tiến thật $\Delta = 0,8$ mAP. Với kiểm định hai mẫu, xấp xỉ chuẩn cho cỡ mẫu mỗi nhóm

$$n \approx \frac{2(z_{1-\alpha/2} + z_{1-\beta})^2 \sigma^2}{\Delta^2} = \frac{2(1,96 + 0,84)^2 (0,6)^2}{(0,8)^2} \approx 8,8,$$

tức khoảng chín seed mỗi cấu hình, mười tám lần chạy cho một so sánh. Bây giờ đi ngược lại, với ba seed mỗi bên. Sai số chuẩn của hiệu số khi đó là $0,6\sqrt{2/3} \approx 0,49$. Hiệu số thật 0,8 chỉ nằm cách ngưỡng 1,96 đúng 0,33 độ lệch. Xác suất phát hiện được rơi vào khoảng 0,35 theo xấp xỉ chuẩn, và còn thấp hơn nếu tính bằng phân phối t với bậc tự do nhỏ. Nói cách khác, ở ba seed bạn bỏ sót khoảng hai phần ba số cải tiến có thật. Hình 2.2 cho thấy vì sao: hai phân phối một-lần-chạy chồng lên nhau gần hết, và chỉ khi lấy trung bình nhiều seed thì chúng mới tách ra.



Hình 2.2. Với $\sigma = 0,6$ mAP giữa các seed, một cải tiến thật 0,8 mAP nằm gần như hoàn toàn trong vùng chồng lấn của hai phân phối một-lần-chạy. Lấy trung bình chín seed thu hẹp độ lệch xuống $\sigma/3 = 0,2$ và tách hai phân phối ra. Đây là lý do hình học của công thức cỡ mẫu ở trên.

B. Hai cách rẻ hơn là tăng số seed

Có hai cách rẻ hơn là tăng số seed. Thứ nhất, **so sánh theo cặp** (*paired comparison*): hai mô hình được đánh giá trên đúng cùng một tập test, nên phần phương sai do độ khó của từng mẫu là chung và biến mất khi lấy hiệu số. Bootstrap theo cặp làm thế này: lấy mẫu lại các ảnh, rồi tính hiệu số metric trên mỗi lần lấy mẫu. Cách đó cho khoảng tin cậy chặt hơn hẳn so với việc so hai con số tổng, và không tốn thêm một lần huấn luyện nào. Thứ hai, báo cáo **cỡ hiệu ứng** (*effect size*) chứ không chỉ p-value: “có ý nghĩa thống kê” và “đáng để triển khai” là hai mệnh đề khác nhau, và với tập test đủ lớn thì mọi khác biệt đều có ý nghĩa thống kê. Khi phải so nhiều phương pháp trên nhiều tập dữ liệu thì các kiểm định phi tham số theo hạng là lựa chọn an toàn hơn kiểm định t [10].

C. So sánh bội và rò rỉ tập test

Vấn đề còn lại là **so sánh bội** (*multiple comparisons*), và nó nghiêm trọng hơn vẻ ngoài. Nếu bạn thử 20 biến thể đều vô dụng như nhau và báo cáo cái tốt nhất, thì cái tốt nhất đó là cực đại của 20 mẫu độc lập từ cùng phân phối nhiễu. Kỳ vọng của cực đại 20 mẫu chuẩn tắc vào khoảng 1,9; với $\sigma = 0,6$ mAP, đó là một “cải tiến” khoảng 1,1 mAP sinh ra hoàn toàn từ nhiễu. Con số này lớn hơn phần lớn cải tiến được công bố trong các bảng detection, và nó giải thích vì sao thứ tự thao tác — chọn trên val, báo cáo trên test đúng một lần — không phải hình thức. Cùng cơ chế đó vận hành chậm hơn nhưng chắc chắn hơn khi bạn dùng lại một tập test suốt nhiều tháng: mỗi quyết định thiết kế dựa trên điểm test là một lần rò rỉ nhỏ. Bằng chứng thực nghiệm ở đây tinh tế: khi dựng lại một tập test mới cho ImageNet và CIFAR-10 theo đúng quy trình gốc, mọi mô hình đều tụt điểm đáng kể, nhưng *thứ hạng giữa chúng gần như được bảo toàn* [11]. Diễn giải đúng là khoảng

cách tuyệt đối không đáng tin trong khi so sánh tương đối thì bền hơn ta lo — chứ không phải “rò rỉ thích ứng không tồn tại”.

CẠM BẤY

Hai dạng ô nhiễm đặc thù của deep learning mà kỷ luật SLAM không dạy bạn. Một, mô hình nền được pretrain trên dữ liệu web quy mô lớn có thể đã nhìn thấy chính ảnh trong tập test của bạn; khi đó “zero-shot” là một tuyên bố không kiểm chứng được nếu không dò trùng lặp. Hai, chính tập test có lỗi nhãn ở tỉ lệ không nhỏ trong các benchmark phổ biến, và ở vùng chênh lệch dưới một điểm thì thứ hạng có thể đảo khi sửa nhãn [12]. Cả hai đều không hiện ra dưới dạng phương sai giữa seed, nên chạy thêm seed không phát hiện được chúng.

D. Phi tất định trong deep learning

Cuối cùng là phần bạn đã sống qua nhưng ở dạng khác: nguồn phi tất định. Các nguồn chính trong deep learning:

- **Chọn thuật toán** — *autotune* của *cuDNN* chọn kernel tích chập khác nhau giữa các lần chạy tùy theo đo thời gian tức thời.
- **Cộng không có thứ tự** — các kernel dùng `atomicAdd` ở **lan truyền ngược** (*backpropagation*) cộng dồn theo thứ tự do lịch chạy quyết định; phép cộng dấu phẩy động không kết hợp, nên tổng đổi ở chữ số cuối. Đây đúng là cơ chế đã sinh ra chênh lệch một inlier trong hệ SLAM của bạn, chỉ khác là ở đây nó nằm trong thư viện chứ không nằm trong code bạn sửa được.
- **Độ chính xác rút gọn** — *TF32* và các đường *mixed precision* trên tensor core cho kết quả khác FP32 và khác nhau giữa các thế hệ GPU.
- **Thứ tự dữ liệu** — dataloader nhiều *worker* trả *batch* theo thứ tự hoàn thành, và mỗi worker có dòng số ngẫu nhiên riêng cho *augmentation*.

Bật chế độ tất định xử lý được vài nguồn trong số đó, phải trả bằng tốc độ, và *vẫn* không đảm bảo hai máy khác GPU cho cùng kết quả. Kết luận thực dụng — giống hết kết luận bạn đã rút ra cho harness của mình, chỉ khác là ở đây không có bug nào để sửa — là mục tiêu đúng phải chuyển từ tái lập từng chữ số sang tái lập phân phối: cùng seed, cùng máy thì khớp; khác máy thì trung bình và độ lệch chuẩn phải nằm trong khoảng tin cậy của nhau. Phần hạ tầng để làm được điều đó nằm ở D/22-tai-lap-va-mlops.

→ **cùng nguyên lý với** D/22 — cùng lý do phải gắn mỗi lần chạy với một commit, một config và một phiên bản dữ liệu: cái không ghi lại được thì không so lại được

2.5 Nhóm D — phân tích lỗi (Error Analysis)

A. Một trăm lỗi gán tay

Đến đây bạn đã biết một cải tiến có thật hay không. Câu hỏi tiếp theo đắt hơn nhiều: nên đầu tư vào đâu. Không có công cụ tự động nào thay thế được việc **phân loại một trăm lỗi bằng tay**. Quy trình là lấy ngẫu nhiên một trăm mẫu bị sai, xem từng mẫu, dựng dần một taxonomy lỗi từ chính chúng, đếm tần suất mỗi loại, rồi ước lượng trần cải thiện nếu loại đó biến mất hoàn toàn. Cỡ mẫu một trăm không tùy tiện: với một loại chiếm 20%, sai số chuẩn của tỉ lệ là $\sqrt{0,2 \cdot 0,8/100} = 0,04$, tức là đủ để xếp hạng các loại lỗi lớn nhưng không đủ để phân biệt hai loại chênh nhau vài điểm

phần trăm. Điều đó là đủ, vì mục đích ở đây là chọn ra một hoặc hai loại đáng theo đuổi, chứ không phải xếp hạng chính xác.

B. Phân rã tự động và phân tích theo lát cắt

Phân rã tự động bổ sung chứ không thay thế bước thủ công. Với detection, các công cụ tách lỗi thành nhầm lớp, lệch vị trí, trùng lặp, bỏ sót và nền bị nhận nhầm; cách tính đúng đắn là đo phần mAP lấy lại được nếu sửa riêng từng loại, thay vì chỉ đếm số lượng [13], [14]. Giá trị của cách phân rã này là mỗi loại lỗi dẫn tới một cách sửa *khác nhau*:

- **Lệch vị trí** — hàm mất mát hồi quy hộp và độ phân giải đặc trưng.
- **Nhầm lớp** — dữ liệu và ranh giới định nghĩa lớp.
- **Bỏ sót** — quy tắc gán mẫu dương và ngưỡng điểm.
- **Trùng lặp** — hậu xử lý.

Bên cạnh đó là **phân tích theo lát cắt** (*slice analysis*): chia tập đánh giá theo các tiêu chí có nghĩa — kích thước vật thể, ánh sáng, camera, nhóm người — và đọc metric trên từng lát. Đây là nơi con số tổng nói dối nhiều nhất, vì một mô hình có thể giữ nguyên điểm tổng trong khi tụt sâu ở một lát chiếm 5% dữ liệu nhưng lại là 100% các trường hợp quan trọng. Với hệ thống robot, các lát tự nhiên là chuyển động nhanh, ánh mờ, bề mặt ít texture — và bạn đã biết rằng một con số trung bình trên toàn EuRoC che giấu hoàn toàn việc một chuỗi cụ thể sập.

C. Bốn ô hành động

Bước cuối khép vòng: mỗi loại lỗi phải được gán vào đúng một trong bốn ô hành động.

- **Sửa bằng dữ liệu** — lỗi tập trung ở một vùng phân phối thiếu mẫu, hoặc ở ranh giới định nghĩa lớp mà chính người gán nhãn cũng bất đồng.
- **Sửa bằng hàm mất mát (*loss function*)** — lỗi có dạng lệch hệ thống mà hàm mất mát hiện tại không phạt, ví dụ lệch vị trí hộp hoặc mất cân bằng lớp.
- **Sửa bằng kiến trúc** — lỗi do thiếu thông tin ở đầu vào của tầng quyết định: độ phân giải đặc trưng, trường tiếp nhận, phạm vi ngưỡng cảnh.
- **Sửa bằng ràng buộc hệ thống** — lọc theo thời gian, luật hậu kiểm, hoặc từ chối trả lời. Đây là ô mà một kỹ sư robot quen nhất, và cũng là ô rẻ nhất. Nó chỉ giấu lỗi đi chứ không sửa mô hình. Nhưng với các lỗi hiếm mà hậu quả lớn, giấu đúng cách lại chính là đáp án; xem [E/25-do-tin-cay-va-van-hanh](#).

Việc gán này phải làm *trước* khi viết một dòng code, vì nó là chỗ duy nhất trong quy trình mà bạn còn ra quyết định bằng suy nghĩ thay vì bằng quán tính. Kinh nghiệm thực hành lặp lại ở nhiều nhóm — không phải một kết quả đo được — là ô thứ nhất thắng nhiều hơn người ta tưởng và ô thứ ba thắng ít hơn người ta tưởng.

I 2.6 Nhóm E — báo cáo và đọc ablation của người khác (Reporting)

↔ **tiền đề** B/06 — taxonomy lỗi dựng từ một trăm mẫu gán tay chính là đầu vào của vòng lỗi → dữ liệu ở chương chất lượng dữ liệu

A. Bảng ablation đạt chuẩn

Một bảng ablation đạt chuẩn phải mang theo bốn thứ mà phần lớn bảng đã công bố không có:

- **Số seed và độ lệch chuẩn** — thiếu hai con số này thì mỗi dòng chỉ là một mẫu duy nhất.
- **Ngân sách tính chỉnh của từng dòng** — mỗi dòng đã được thử bao nhiêu cấu hình.
- **Tập nào** — nói rõ ablation chạy trên validation chứ không phải trên test.
- **Thay đổi ngầm kéo theo** — bỏ một thành phần thường đổi luôn số tham số, thời gian huấn luyện và learning rate tối ưu; phải ghi rõ cái nào đã được re-tune.

Thiếu bốn thứ này thì bảng vẫn đọc được như một bảng, nhưng nó không còn là bằng chứng.

B. Ghi cả kết quả âm

Điểm thứ hai quan trọng hơn và ít được làm nhất: **ghi lại cả kết quả âm**. Một ý tưởng đã thử và không hiệu quả có giá trị gần bằng một ý tưởng hiệu quả, vì nó thu hẹp không gian tìm kiếm cho chính bạn sáu tháng sau và cho người tiếp theo. Bạn đã có sẵn thói quen này ở phía SLAM — vài kết luận NO-GO được ghi lại kèm điều kiện thí nghiệm — và nên giữ nguyên nó khi chuyển sang deep learning, với một điều chỉnh: kết quả âm chỉ có giá trị khi nó *ghi kèm phạm vi hiệu lực*. “Ý tưởng X không hiệu quả” là một câu mê tín; “ý tưởng X không cho cải tiến vượt sà nhiều $\pm 0,6$ mAP trên tập này, ở quy mô này, với ngân sách tune này, qua ba seed” là một kết quả. Mệnh đề thứ hai cho bạn biết điều kiện nào thay đổi thì đáng thử lại. Trong deep learning điều kiện thay đổi liên tục, nên một NO-GO không ghi điều kiện sẽ chặn bạn khỏi đúng cơ hội về sau.

C. Đọc ablation của người khác

Điểm thứ ba là mặt đọc, chính là chương 23 áp vào một bảng cụ thể: ablation chạy trên tập nào, bao nhiêu seed, baseline có được tune không, và thành phần bị ablate có kéo theo thay đổi ngầm nào không. Bốn câu hỏi đó loại được phần lớn bảng ablation rỗng, nhanh hơn mọi cách đọc khác.

CẦN NHỚ

Một kết luận âm được ghi lại *kèm điều kiện thí nghiệm* là một kết quả, không phải một thất bại. Nó là thứ duy nhất trong công việc thí nghiệm không bị mất giá theo thời gian, miễn là bạn ghi đủ điều kiện để biết khi nào nên thử lại. [T2]

2.7 Giới hạn và trường hợp hỏng

Kỷ luật trên có một chi phí thẳng thắn: chín seed nhân một thiết kế giai thừa là ngân sách rất ít nhóm trả nổi. Thoả hiệp thực dụng là phân tầng — sàng nhanh bằng một seed để loại ứng viên vô vọng, rồi chỉ chạy đầy đủ cho cấu hình sống sót. Nhưng vòng sàng đã tiêu một phần “ngân sách tin cậy”: cấu hình sống sót được chọn bởi nhiều cũng nhiều như bởi chất lượng, nên con số của vòng đầy đủ vẫn thiên lệch lên trên nếu dùng lại cùng tập dữ liệu. Cách sửa rẻ nhất là tách tập sàng khỏi tập xác nhận.

Chế độ hỏng thứ hai mang tính khái niệm. Ablation giả định các thành phần *tách rời được*, tức bỏ một thành phần không làm hệ thống chuyển sang chế độ vận hành khác. Giả định này đúng với phần lớn mạng feed-forward và sai với hệ có vòng phản hồi: trong một pipeline SLAM, chất lượng bản đồ chỉ phối việc chọn keyframe và ngược lại, nên bỏ một thành phần trong vòng đó cho ra một hệ thống khác chứ không phải “hệ thống cũ trừ thành phần”. Khi ấy “đóng góp của

thành phần” không còn là đại lượng xác định, và câu hỏi đúng đổi thành: hệ chuyển sang chế độ nào, và chế độ đó tốt hay xấu hơn ở đâu.

Chế độ hỏng thứ ba là chính sự tự tin do thống kê mang lại: một khoảng tin cậy hẹp chỉ nói rằng bạn đã đo chính xác *trên phân phối của tập đánh giá*. Nếu tập đó không đại diện cho môi trường triển khai, mọi kỷ luật ở chương này chỉ giúp bạn sai một cách rất chính xác — địa hạt của B/08-dịch-chuyen-phan-phoi.

2.8 Thực hành

Repo và công cụ. [dbolya/tide](#) để phân rã lỗi detection; [voxel51/fiftyone](#) để cắt lát, xem lỗi và so hai mô hình trên cùng mẫu; [wandb/wandb](#) sweeps và [optuna](#) để quản lý nhiều lần chạy có kỷ luật; `scipy.stats` cộng một hàm bootstrap tự viết cho kiểm định theo cặp; [facebookresearch/hydra](#) multirun để một thay đổi luôn tương ứng một config diff. Nên đọc thêm phần ablation của những paper có kỷ luật thí nghiệm tốt — dòng ConvNeXt, dòng “bag of tricks”, các paper về công thức huấn luyện — vì chúng dạy phương pháp rõ hơn hẳn các paper kiến trúc. Danh sách công cụ này chốt tại tháng 9/2026 và sẽ cũ trong sáu đến mười hai tháng; các nguyên tắc trong chương thì không.

Câu hỏi kiểm tra hiểu. Bạn bỏ một module và mAP giảm 1,5 — nêu bốn cách giải thích khác ngoài “module đó quan trọng”. Nếu phương sai giữa seed là $\pm 0,6$ mAP, cần bao nhiêu lần chạy để tin một cải tiến 0,8 mAP, và vì sao so sánh theo cặp lại rẻ hơn cách đó? Oracle experiment nào bạn nên chạy đầu tiên cho hệ thống hiện tại của mình, và nó chặn trên cái gì?

Mini project. Lấy một repo có sẵn, *viết trước* dự đoán cho năm ablation kèm con số bạn nghĩ sẽ thấy, chạy mỗi cái ba seed, rồi đối chiếu: chỗ nào đúng, chỗ nào sai, trực giác nào cần sửa. Cột “dự đoán” là phần quan trọng nhất, vì nó là thứ duy nhất biến kết quả thành thông tin về mô hình tư duy của bạn.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
•• Vì sao ba loại câu hỏi thí nghiệm cần ba thiết kế?	Vì chúng hỏi những thứ khác nhau. “Có hoạt động không” cần kiểm soát chặt và nhiều seed; “vì sao hoạt động” cần ablation có re-tune; “hoạt động tới đâu” cần nhiều điểm dọc một trục, ít seed mỗi điểm.
••• Baseline	Phương pháp đối chứng mà kết quả mới phải vượt qua. Một baseline chưa được tinh chỉnh tử tế làm vô nghĩa mọi con số phía sau, và đó là cách tự lừa mình phổ biến nhất vì nó rẻ.
•• Thiết kế giai thừa	Chạy mọi tổ hợp bật/tắt của các thành phần thay vì đổi từng cái một. Với ba thành phần là 8 cấu hình so với 4, tức gấp đôi chi phí, đổi lại bạn thấy được toàn bộ tương tác bậc hai.
••• Leave-one-out	Bỏ từng thành phần khỏi hệ thống đầy đủ rồi đo lại. Nó trả lời “thành phần này còn đáng giữ không”, và chỉ đúng khi phần còn lại vẫn ở điểm vận hành tốt.
•• Add-one-in	Thêm từng thành phần vào baseline trần. Nó trả lời “thành phần này tự nó có giá trị không”; khi có tương tác, con số của nó tách khỏi leave-one-out, và khoảng cách ấy chính là lượng tương tác.
••• Vì sao ablation phải re-tune?	Vì bỏ một thành phần ổn định hoá thì learning rate tối ưu của phần còn lại nhỏ đi. Giữ nguyên learning rate cũ là đo “thành phần đó cộng một điểm vận hành đã sai”, và sai lệch luôn thổi phồng.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●● Sanity ablation	Nhóm phép thử thô: thay đầu vào bằng nhiễu, xáo trộn nhãn, thay module bằng hằng số, cắt bỏ một modality. Rẻ nhất mà phá được nhiều giả thuyết nhất, nên chạy trước mọi ablation tinh vi.
●●● Xáo trộn nhãn mà điểm đánh giá không giảm — nghĩa là gì?	Hoặc có rò rỉ dữ liệu, hoặc thành phần đang xét vô dụng. Phân biệt với hiện tượng bình thường: train loss vẫn giảm khi xáo trộn nhãn là chuyện thường, chỉ có điểm trên tập đánh giá mới là bằng chứng.
●●● Oracle experiment	Thay một khối bằng chân trị rồi đo hệ thống tổng thể, để biết trần đóng góp của khối đó. Nếu detector hoàn hảo chỉ nâng hệ thống 2%, mọi công sức tối ưu detector đều bị chặn trên bởi 2%.
●●● Sàn nhiễu	Độ lệch chuẩn giữa các seed của <i>cùng một</i> cấu hình. Nó là đơn vị đo của mọi so sánh: chênh lệch nhỏ hơn nó thì chưa nói được gì, dù bạn lặp lại bao nhiêu lần.
●●● Cỡ mẫu tối thiểu	Số seed cần cho mỗi cấu hình, tính từ sàn nhiễu và độ lớn cải tiến muốn phát hiện: $n \approx 2(1,96 + 0,84)^2 \sigma^2 / \Delta^2$. Với $\sigma = 0,6$ và $\Delta = 0,8$ là khoảng chín seed mỗi bên.
●● So sánh theo cặp	Đánh giá hai mô hình trên đúng cùng tập test rồi lấy hiệu số trên từng mẫu. Phần phương sai do độ khó từng ảnh là chung nên biến mất, và bootstrap theo cặp cho khoảng tin cậy chặt hơn mà không tốn thêm lần huấn luyện nào.
●●● So sánh bội	Thử nhiều biến thể rồi báo cáo cái tốt nhất. Kỳ vọng của cực đại hai mươi lần rút đã cỡ 1,9 độ lệch chuẩn, nên với $\sigma = 0,6$ mAP thì “cải tiến” 1,1 điểm có thể là nhiễu thuần.
●●● Cải thiện 1,1 mAP sau khi thử hai mươi cấu hình — nghĩa là gì?	Nhiều khả năng là nhiễu. Cách phân biệt: chạy lại đúng cấu hình thắng đó với vài seed mới trên một tập giữ riêng; nếu cải tiến bốc hơi thì bạn vừa đo cỡ mẫu của cuộc tìm kiếm.
●● Rò rỉ tập test	Tập test mất dần tư cách trọng tài vì bị dùng lại hàng trăm lần, hoặc vì nó nằm sẵn trong dữ liệu pretraining. Nó không hiện ra dưới dạng phương sai giữa seed, nên chạy thêm seed không phát hiện được.
●● Vì sao chế độ tắt định vẫn không tái lập được?	Vì nguồn phi tắt định nằm trong thư viện và phần cứng: cuDNN chọn kernel khác nhau, atomics cộng không theo thứ tự, TF32 khác giữa các đời GPU. Mục tiêu đúng là tái lập phân phối, không phải từng chữ số.
●●● Hai máy khác GPU cho ATE lệch 3 cm trên cùng cấu hình — bug hay không?	So với sàn nhiễu đã đo trước đó. Nếu 3 cm nằm trong khoảng dao động giữa các seed thì đó là phi tắt định của thư viện, không phải bug. Nếu vượt hẳn, hãy nghi khác phiên bản thư viện, khác dữ liệu, hoặc một cấu hình chưa được ghi.
●● Vì sao công cụ tự động không thay được gán tay?	Vì công cụ chỉ đếm được các loại lỗi đã có tên sẵn. Gán tay là chỗ taxonomy mới sinh ra; sau đó công cụ mới đếm và ước lượng trần cải thiện cho từng loại.
●● Bốn ô hành động	Mỗi loại lỗi phải được gán vào đúng một ô: sửa bằng dữ liệu, bằng hàm mất mát, bằng kiến trúc, hoặc bằng ràng buộc hệ thống. Gán trước khi viết code, vì đó là chỗ duy nhất bạn còn quyết định bằng suy nghĩ.
●●● Kết quả âm	Ghi lại thứ đã thử và không hiệu quả, kèm tập nào, quy mô nào, mấy seed, sàn nhiễu bao nhiêu. Thiếu điều kiện thì nó thành mê tín và sẽ chặn bạn khỏi đúng cơ hội sáu tháng sau.

●●● phải thuộc ●● nên nắm ● biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
dbolya/tide [13]	Phân rã lỗi detection theo loại, mỗi loại kèm phần mAP lấy lại được nếu sửa riêng nó — chính ý tưởng trần cải thiện theo loại lỗi	Kế thừa phân tích chẩn đoán của Hoiem [14]
voxel51/fiftyone	Cắt lát tập đánh giá và so hai mô hình trên cùng mẫu — hiện thực của phân tích theo lát cắt và so sánh theo cặp	Hạ tầng; bền hơn tên mô hình
Dòng ConvNeXt [3]	Cả paper là một chuỗi ablation recipe-so-với-kiến trúc có kiểm soát	Nên đọc phần ablation hơn phần kiến trúc
Reproducibility checklist (NeurIPS, ICLR)	Ép khai báo số seed, độ lệch chuẩn và ngân sách tính toán — đúng bốn yêu cầu của một bảng ablation đạt chuẩn	Thông lệ từ khoảng 2019
Benchmark harness EuRoC của chính bạn	Quy tắc ba lần chạy, chế độ lockstep để cô lập phi tắt định, các kết luận NO-GO có ghi điều kiện — nhóm C và nhóm E ở dạng thực địa	Ví dụ nội bộ; số liệu nằm trong ghi chú dự án

MINI PROJECT SLAM · Đo sần nhiễu của chính harness EuRoC của bạn · 1-2 ngày

Mục tiêu	biến câu “một lần chạy không nói lên điều gì” thành một con số cụ thể cho hệ SLAM của bạn. Từ con số đó suy ra số lần chạy tối thiểu cho mọi so sánh về sau.
Dữ liệu	hai chuỗi EuRoC có độ khó khác nhau (ví dụ một chuỗi dễ và V2_03), cấu hình mặc định, không đổi bất cứ tham số nào giữa các lần chạy.
Các bước	chạy <i>cùng một</i> cấu hình mười lần trên mỗi chuỗi; với mỗi lần ghi ATE bằng evo cùng tỉ lệ khung hình bám được; lập bảng mười dòng, tính trung bình và độ lệch chuẩn σ riêng cho từng chuỗi; rồi áp công thức cỡ mẫu $n \approx 2(z_{1-\alpha/2} + z_{1-\beta})^2 \sigma^2 / \Delta^2$ cho hai mức Δ bạn quan tâm, chẳng hạn 1 cm và 3 cm.
Xong khi	bạn phát biểu được, kèm bảng số chống lưng, câu “trên chuỗi này, chênh lệch ATE nhỏ hơn X cm giữa hai cấu hình là nhiễu”, và nói được số lần chạy tối thiểu cần cho mỗi Δ . Kiểm tra chéo: σ của chuỗi khó phải lớn hơn hẳn σ của chuỗi dễ; nếu không, harness của bạn còn một nguồn phi tắt định chưa lộ.
Nối với	B/07 cho giao thức đánh giá và cách chọn chuỗi; D/22 cho việc gắn mỗi lần chạy với một commit và một config; E/25 sẽ dùng lại chính bảng số này khi đặt ngưỡng theo chi phí.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- Vì con số bạn báo cáo là *cực đại* của hai mươi mẫu chứ không phải một mẫu, và kỳ vọng của cực đại tăng theo cỡ mẫu. Với 20 mẫu chuẩn tắc, kỳ vọng cực đại khoảng 1,9; nhân với $\sigma = 0,6$ ra $\approx 1,1$ điểm sinh ra hoàn toàn từ nhiễu.
- Với $\sigma = 0,6$ và cải tiến thật 0,8, cần khoảng chín seed mỗi bên. Ở ba seed, sai số chuẩn của hiệu số là $0,6\sqrt{2/3} \approx 0,49$, nên xác suất phát hiện chỉ khoảng 0,35 — bỏ sót chừng hai phần ba số cải tiến có thật. Ba là sần để phát hiện *sự bất định*, không phải mức đủ để *kết luận*.
- Bốn cách: chênh lệch nằm trong sần nhiễu; learning rate không được re-tune sau khi bỏ module; việc bỏ module kéo theo một thay đổi ngầm (số tham số, thời gian huấn luyện, đường dữ liệu); và module chỉ quan trọng ở *quy mô dữ liệu này*. Cách thứ hai luôn đẩy kết luận về cùng một hướng — thổi phồng — nên nó không tự triệt tiêu khi lặp lại nhiều lần.
- Không còn lý do nào: mọi công sức tối ưu detector bị chặn trên bởi 2%. Bạn biết con số ấy trước bằng một oracle experiment — thay detector bằng chân trị rồi đo hệ thống tổng thể — và đó là thí nghiệm nên chạy *đầu tiên* chứ không phải cuối cùng.
- Vì “X không hiệu quả” không cho biết điều kiện nào thay đổi thì đáng thử lại. Trong deep learning điều kiện đổi liên tục: quy mô dữ liệu, mô hình nền, ngân sách tính chính. Một NO-GO không ghi điều kiện sẽ chặn bạn khỏi đúng cơ hội về sau, trong khi không ghi gì thì ít nhất bạn còn thử lại.

NÓI LẠI THẬT ĐƠN GIẢN

Bạn mua một cái cân trong nhà tắm. Bước lên ba lần trong cùng một phút, nó hiện ba số khác nhau, lệch nhau chừng ba lạng. Bây giờ bạn ăn kiêng một tuần rồi cân lại: nhẹ hơn hai lạng. Bạn có giảm cân thật không? Không ai trả lời được, vì hai lạng còn nhỏ hơn mức cái cân tự dao động. Câu hỏi phải hỏi trước là: cái cân này dao động bao nhiêu? Cân mười lần trong cùng một buổi sáng, ghi cả mười số lại, rồi mới biết chênh lệch bao nhiêu mới đáng nói.

Từ đó ra hai điều. Thứ nhất, muốn phát hiện một thay đổi nhỏ hơn mức dao động, bạn phải cân nhiều lần hơn — và số lần ấy tính ra được chứ không phải chọn theo thói quen. Thứ hai, nếu bạn thử hai mươi thực đơn rồi chỉ kể lại cái cho số đẹp nhất, thì phần lớn cái “đẹp nhất” ấy là do cái cân chứ không do thực đơn.

Cái giá phải trả: cân mười lần thì tốn gấp mười lần công. Và bạn phải chịu khó ghi lại cả những tuần không giảm được lạng nào, kèm điều kiện đầy đủ — ăn gì, cân lúc nào, cân nào — vì chính những đồng đó ngăn bạn thử lại đúng thực đơn ấy vào tháng sau.

Chỗ không rút gọn được: phần tính toán. Rút gọn cả chương thành “chạy nhiều lần cho chắc” nghe rất hợp lý nhưng làm mất đúng chỗ có giá trị: số lần chạy không phải một thói quen mà là kết quả của một phép tính, suy ra từ mức dao động bạn đã đo và từ độ lớn cải thiện bạn muốn phát hiện. Ba lần là thói quen; chín lần là một phép tính — và khác biệt giữa hai con số đó là bạn nói được mình đang chấp nhận bỏ sót bao nhiêu.

Một câu: Trước khi hỏi câu hình nào tốt hơn, hãy hỏi cái cân của bạn dao động bao nhiêu.

2.9 Kết nối

B/07-chia-du-lieu-va-danh-gia là tiền đề trực tiếp: cách chia dữ liệu sai thì mọi thống kê ở đây chỉ đo rất chính xác một đại lượng vô nghĩa. Phần đọc *ý nghĩa* của từng con số — precision, recall, F1, đường ROC và PR, hiệu chỉnh, và bảng chẩn đoán ngược từ triệu chứng về nguyên nhân — cũng nằm ở B/07-chia-du-lieu-va-danh-gia. E/23-doc-paper là mặt đối xứng: tiêu chí bạn dùng để nghi ngờ ablation của người khác chính là tiêu chí bạn phải tự thoả. D/22-tai-lap-va-mlops là hạ tầng khiến kỷ luật này khả thi — một thay đổi bằng một config diff, mỗi lần chạy gắn với một commit và một phiên bản dữ liệu. B/08-dich-chuyen-phan-phoi là giới hạn ngoài: nó cho thấy khi nào một kết luận đúng trên tập đánh giá vẫn không chuyển được sang thực tế. Cuối cùng, E/26-lo-trinh-hoc cho biết chương này nằm ở đâu trong từng lộ trình theo mục tiêu — nó là chương trung tâm của mọi lộ trình có chữ “đánh giá” trong mục tiêu.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao sai lệch do không re-tune sau khi bỏ một thành phần lại luôn cùng một chiều, và vì sao chạy thêm seed không sửa được sai lệch đó?
2. Leave-one-out và add-one-in cho hai con số khác nhau cho cùng một thành phần. Điều đó nói lên gì về hệ thống, và bạn báo cáo con số nào?
3. Với $\sigma = 0,6$ và 20 biến thể vô dụng như nhau, “cải tiến” tốt nhất bạn kỳ vọng thấy là bao nhiêu? Con số đó so với các cải tiến thường được công bố thì thế nào?
4. Bật chế độ tắt định của framework rồi mà hai máy vẫn cho kết quả khác nhau: ba nguyên nhân là gì, và vì sao mục tiêu “tái lập từng chữ số” là mục tiêu sai?
5. Một oracle experiment cho trần lạc quan giả trong trường hợp nào — cụ thể, chuyện gì xảy ra với phân phối đầu vào của khối đứng ngay sau oracle?
6. Vì sao một kết luận NO-GO không ghi điều kiện thí nghiệm lại có hại hơn là không ghi gì?
7. Ablation giả định điều gì về cấu trúc hệ thống, và giả định đó hỏng thế nào với một hệ có vòng phản hồi?

Nguồn

- [1] S. Keshav, *How to Read a Paper*, ACM SIGCOMM Computer Communication Review, 2007.
- [2] S. Zhang, C. Chi, Y. Yao, et al., “Bridging the Gap Between Anchor-Based and Anchor-Free Detection via Adaptive Training Sample Selection,” in *CVPR*, 2020.
- [3] Z. Liu, H. Mao, C.-Y. Wu, et al., “A ConvNet for the 2020s,” in *CVPR*, 2022.
- [4] K. Musgrave, S. Belongie, and S.-N. Lim, “A Metric Learning Reality Check,” in *ECCV*, 2020.
- [5] M. Ferrari Dacrema, P. Cremonesi, and D. Jannach, “Are We Really Making Much Progress? A Worrying Analysis of Recent Neural Recommendation Approaches,” in *RecSys*, 2019.
- [6] P. Henderson, R. Islam, P. Bachman, et al., “Deep Reinforcement Learning That Matters,” *AAAI*, 2018.
- [7] J. Bergstra and Y. Bengio, “Random Search for Hyper-Parameter Optimization,” in *JMLR*, 2012.
- [8] P. Goyal, P. Dollár, R. Girshick, et al., “Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour,” *arXiv:1706.02677*, 2017.
- [9] C. Zhang, S. Bengio, M. Hardt, et al., “Understanding deep learning requires rethinking generalization,” in *ICLR*, 2017.
- [10] J. Demšar, “Statistical Comparisons of Classifiers over Multiple Data Sets,” *JMLR*, vol. 7, pp. 1–30, 2006.
- [11] B. Recht, R. Roelofs, L. Schmidt, et al., “Do ImageNet Classifiers Generalize to ImageNet?” In *ICML*, 2019.
- [12] C. G. Northcutt, A. Athalye, and J. Mueller, “Pervasive Label Errors in Test Sets Destabilize Machine Learning Benchmarks,” in *NeurIPS Datasets and Benchmarks*, 2021.
- [13] D. Bolya, S. Foley, J. Hays, et al., “TIDE: A General Toolbox for Identifying Object Detection Errors,” in *ECCV*, 2020.
- [14] D. Hoiem, Y. Chodpathumwan, and Q. Dai, “Diagnosing Error in Object Detectors,” in *ECCV*, 2012.
- [15] L. van der Maaten and G. Hinton, “Visualizing Data Using t-SNE,” *JMLR*, vol. 9, pp. 2579–2605, 2008.
- [16] L. McInnes, J. Healy, and J. Melville, “UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction,” *arXiv:1802.03426*, 2018.
- [17] R. R. Selvaraju, M. Cogswell, A. Das, et al., “Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization,” in *ICCV*, 2017.
- [18] J. Adebayo, J. Gilmer, M. Muehly, et al., “Sanity Checks for Saliency Maps,” in *NeurIPS*, 2018.
- [19] C. Guo, G. Pleiss, Y. Sun, et al., “On Calibration of Modern Neural Networks,” in *ICML*, 2017.
- [20] Y. Ovadia, E. Fertig, et al., “Can You Trust Your Model’s Uncertainty?” In *NeurIPS*, 2019.
- [21] Y. Gal and Z. Ghahramani, “Dropout as a Bayesian Approximation: Representing Model Uncertainty in Deep Learning,” in *ICML*, 2016.

- [22] B. Lakshminarayanan, A. Pritzel, and C. Blundell, "Simple and Scalable Predictive Uncertainty Estimation Using Deep Ensembles," in *NeurIPS*, 2017.
- [23] A. N. Angelopoulos and S. Bates, *Conformal Prediction: A Gentle Introduction*. Foundations and Trends in Machine Learning, 2023.
- [24] D. Hendrycks and T. Dietterich, "Benchmarking Neural Network Robustness to Common Corruptions and Perturbations," in *ICLR*, 2019.
- [25] D. Hendrycks and K. Gimpel, "A Baseline for Detecting Misclassified and Out-of-Distribution Examples in Neural Networks," in *ICLR*, 2017.
- [26] M. Mitchell, S. Wu, et al., "Model Cards for Model Reporting," in *FAT**, 2019.

Trực quan hoá, độ tin cậy và vận hành (Visualization, Reliability, Operations)

PHỤ THUỘC C/12-co-che-huan-luyen/01-co-hoc-cua-gradient (mọi công cụ ở nhóm A chỉ là gradient theo đầu vào thay vì theo trọng số); B/08-dich-chuyen-phan-phoi (bất định và drift là hai mặt của cùng một hiện tượng); E/24-thi-nghiem-va-ablation (kỷ luật đo đạc — chương này thêm câu hỏi “khi nào được phép tin một con số đơn lẻ”).

TÓM TẮT

Chương này trả lời câu hỏi vận hành: *khi nào tôi được phép tin đầu ra của mô hình, và làm gì khi không tin được*. Đọc xong bạn biết vì sao một bản đồ nhiệt đẹp gần như không chứng minh điều gì và phải kiểm chứng bằng can thiệp nào; bạn hiệu chỉnh được xác suất đầu ra và đặt được ngưỡng từ chối trả lời bằng chi phí lỗi thật chứ bằng cảm tính; và bạn thiết kế được đường giám sát khi không có nhãn ở môi trường triển khai. Nếu chỉ nhớ một điều: một ước lượng bất định chỉ có giá trị khi nó nổi vào một *hành động khác* — từ chối trả lời, chuyển cho người, hoặc hạ mức tự chủ; bất định không đối hành vi là bất định trang trí.

KIẾN THỨC CẦN CÓ TRƯỚC

- **logit và softmax** — logit là vector điểm số thô mà mạng xuất ra trước bước chuẩn hoá; softmax biến nó thành các số dương cộng lại bằng 1, trông như xác suất. “Trông như” là chỗ mấu chốt: tổng bằng 1 không đảm bảo con số đó phản ánh tần suất đúng thật.
- **độ tin cậy (confidence)** — giá trị softmax lớn nhất của một dự đoán, tức là mức mà mô hình “tự nhận” mình chắc chắn.
- **hiệu chỉnh (calibration)** — tính chất mong muốn rằng trong nhóm các lần mô hình nói “chắc 90%”, nó đúng khoảng 90% số lần. Đây là tính chất của *một tập* dự đoán, không phải của một dự đoán đơn lẻ, và nó tách rời với độ chính xác: một mô hình có thể chính xác cao nhưng hiệu chỉnh tệ, và ngược lại.
- **ECE** — viết tắt của *expected calibration error*. Nó chia các dự đoán thành từng khoảng theo độ tin cậy, rồi lấy trung bình khoảng cách giữa độ tin cậy và độ chính xác thật trong mỗi khoảng.
- **tập hiệu chuẩn (calibration set)** — một tập dữ liệu có nhãn, tách riêng khỏi train và test, dùng để học tham số hiệu chỉnh hoặc ngưỡng. Dùng lại tập test làm tập hiệu chuẩn là một dạng rò rỉ.
- **ensemble** — tập hợp k mô hình huấn luyện độc lập, dự đoán bằng cách lấy trung bình. Mức bất đồng giữa các thành viên là một ước lượng bất định rẻ về mặt khái niệm nhưng đắt về mặt tính toán.
- **conformal prediction** — họ phương pháp trả về một *tập* nhãn thay vì một nhãn. Tập đó chứa nhãn đúng với xác suất đặt trước, nhờ một ngưỡng lấy từ phân vị của tập hiệu chuẩn.
- **saliency map** — bản đồ nhiệt gán cho mỗi vùng ảnh một điểm “mức ảnh hưởng tới đầu ra”, thường tính từ gradient của điểm số theo pixel. Nó là một giả thuyết về nơi mô hình nhìn, không phải bằng chứng.
- **drift** — hiện tượng phân phối dữ liệu ở môi trường triển khai trôi khỏi phân phối lúc huấn luyện. Phân biệt hai loại: *dịch chuyển đầu vào* (ảnh đối, quan hệ ảnh-nhãn giữ nguyên) và *dịch chuyển khái niệm* (ảnh trông như cũ nhưng nhãn đúng đã đổi nghĩa).

CÂU HỎI MỞ ĐẦU

- Q1. Grad-CAM chỉ đúng vào con vật trong ảnh. Vì sao điều đó vẫn chưa chứng minh được rằng mô hình nhìn vào con vật — và thí nghiệm một dòng nào bác bỏ được nó?
- Q2. Một mô hình đạt 94% chính xác trong khi độ tin cậy trung bình là 0,99. Con số nào sai, và sửa nó có làm mô hình chính xác hơn không?
- Q3. Nếu bỏ sót đất gấp một trăm lần báo động giả thì ngưỡng tối ưu là 0,01. Vì sao chính cái ngưỡng “đúng” ấy lại có thể khiến hệ thống không triển khai được?
- Q4. Một phương pháp hứa “tập dự đoán chứa nhãn đúng 90% số lần”. Vì sao lời hứa đó vẫn tương thích với việc sai 40% ở đúng lớp bạn quan tâm nhất?

3.1 Đặt vấn đề

Một mô hình phân loại luôn trả về một câu trả lời. Đó là thiết kế, không phải lỗi: hàm *softmax* chuẩn hoá cho tổng bằng một, nên kể cả với một ảnh không thuộc bất kỳ lớp nào nó vẫn cho ra một lớp cùng một con số trông giống xác suất. Với một hệ thống chạy trong phòng thí nghiệm, điều này vô hại vì mọi ảnh đều đến từ tập test. Với một hệ thống chạy thật, nó là chế độ hỏng nguy hiểm nhất, bởi hệ thống sai mà không phát tín hiệu nào cho biết nó đang sai.

Chỗ nổi với nền sẵn có của bạn nằm ngay đây. Một pipeline SLAM không hỏi “đúng bao nhiêu phần trăm”. Nó hỏi “khi nào tôi biết mình đang trôi”. Cả kiến trúc phòng vệ của nó tồn tại để trả lời câu ấy: số inlier, ngưỡng chi-square trên phần dư, cờ mất bám, chế độ định vị lại. Deep learning cho thị giác mang văn hoá ngược lại: tối ưu một con số tổng trên một tập test cố định, hầu như không có cơ chế tự báo sai. *Học xong chương này bạn làm được ba việc mà trước đó không làm được*: đọc một bản đồ nhiệt mà không rút ra kết luận sai; biến điểm số đầu ra thành xác suất dùng được cho quyết định có chi phí bất đối xứng; và viết được tài liệu chế độ hỏng cho một hệ thống thật.

TRỰC GIÁC

Một mạng chưa hiệu chỉnh giống một đồng hồ đo áp suất mà kim đã bị lệch cỡ: nó vẫn chuyển động đúng chiều khi áp suất thay đổi, nên vẫn dùng để *so sánh* được, nhưng con số đọc trên mặt đồng hồ không dùng để *quyết định* được. Hiệu chỉnh là chỉnh lại cỡ; nó không làm cảm biến nhạy hơn, chỉ làm con số trở nên có nghĩa. Và như mọi lần chỉnh cỡ, nó chỉ đúng trong dải điều kiện mà bạn đã chỉnh — ra ngoài dải đó, kim lại lệch.

3.2 Nhìn vào bên trong mô hình (Looking Inside the Model)

Cả nhóm công cụ này chia sẻ đúng một kỹ thuật: lấy *gradient* theo *đầu vào* thay vì theo trọng số. Khi huấn luyện, ta hỏi “đổi trọng số thế nào để mất mát giảm”; khi trực quan hoá, ta hỏi “đổi pixel nào thì điểm số của lớp này đổi nhiều nhất”. Cùng một phép tính, khác biến được coi là tự do — và nhận ra điều đó thì cả nhóm A trở thành một ý tưởng duy nhất thay vì một danh sách kỹ thuật rời rạc.

A. Bốn tầng công cụ

- **Nhìn thẳng trọng số** — filter tầng đầu của một CNN hiện ra dưới dạng bộ lọc biên và đốm màu. Chỉ tầng đầu đọc được bằng mắt; từ tầng hai trở đi, trọng số không còn sống trong không gian ảnh.

- **Chiếu *embedding*** — *t-SNE* [15] và *UMAP* [16] đưa đặc trưng nhiều chiều xuống hai chiều để nhìn cấu trúc cụm.
- **Tối ưu trên pixel** — từ một ảnh nhiễu, đi lên theo gradient của điểm số một lớp, để xem mô hình “muốn thấy” gì ở lớp đó.
- **Quy gán** — *saliency map*, *guided backpropagation*, và *Grad-CAM* [17] trả lời câu “mô hình đang nhìn vào đâu” bằng cách gán điểm cho từng vùng ảnh.

B. Vì sao một bản đồ nhiệt đẹp gần như không chứng minh gì

Đây là chỗ cần nói thẳng là phức tạp chứ không đơn giản hoá. Kết quả đã được công bố và có tính quyết định: một số phương pháp quy gán vẫn cho ra bản đồ trông rất hợp lý ngay cả khi trọng số của mô hình bị ngẫu nhiên hoá, hoặc khi mô hình được huấn luyện trên nhãn xáo trộn [18]. Nếu bản đồ gần như không đổi khi mô hình đã bị phá hỏng, thì bản đồ đó phản ánh cấu trúc của *ảnh đầu vào* và của chính thuật toán quy gán, chứ không phản ánh cái mô hình đã học. Đó là lý do phép thử tính tảo phải chạy trước khi tin bất kỳ bản đồ nào: so bản đồ của mô hình đã huấn luyện với bản đồ của mô hình khởi tạo ngẫu nhiên, và nếu hai bản đồ giống nhau thì phương pháp đó không nói gì về mô hình của bạn.

Cùng lỗi diễn giải xảy ra với phép chiếu hai chiều: trong một bản đồ t-SNE, khoảng cách giữa các cụm và kích thước cụm gần như không mang thông tin, và cấu trúc nhìn thấy phụ thuộc mạnh vào tham số *perplexity*. Kết luận đúng cho cả nhóm: các công cụ này dùng để *đặt giả thuyết*, không đủ để *kết luận*.

C. Che một vùng rồi đo lại

Cách biến một giả thuyết thành bằng chứng là can thiệp thật rồi đo lại. Đó đúng là một ablation như ở chương 24, chỉ khác chỗ bạn cắt bỏ trên *đầu vào* chứ không trên kiến trúc. Nếu bản đồ nhiệt nói mô hình dựa vào một vùng, hãy che vùng đó đi và đo điểm số tụt bao nhiêu; hãy che một vùng ngẫu nhiên cùng diện tích để có nhóm đối chứng; và hãy làm trên đủ mẫu để so được với sàn nhiễu. Đây là chuyển từ quan sát tương quan sang thí nghiệm can thiệp, và nó là ranh giới giữa một hình đẹp trong slide và một kết luận dùng được.

↪ **trường hợp riêng của E/24** — đây đúng là một ablation, chỉ khác là ablate trên đầu vào thay vì trên kiến trúc — nên nó cũng cần nhóm đối chứng và cũng phải vượt sàn nhiễu

Bảng 3.1. Bốn công cụ nhìn vào bên trong mô hình. Cột cuối là cột quyết định: nó cho biết kết luận nào *không* được phép rút ra.

Công cụ	Cơ chế	Nói được gì	Hỏng khi nào
Filter tầng đầu	Nhìn thẳng trọng số	Mô hình có học được bộ lọc biên/màu hợp lý không	Chỉ đọc được ở tầng đầu; im lặng về mọi thứ sâu hơn
t-SNE / UMAP	Bảo toàn lân cận cục bộ	Có tồn tại cấu trúc cụm trong không gian đặc trưng không	Khoảng cách và kích thước cụm vô nghĩa; đổi perplexity đổi câu chuyện
Grad-CAM	Gradient theo đặc trưng, gộp theo kênh	Một giả thuyết về vùng ảnh có ảnh hưởng	Vẫn đẹp với trọng số ngẫu nhiên [18]; độ phân giải thô
Che vùng + đo lại	Can thiệp thật rồi đo	Bằng chứng nhân quả trong phạm vi mô hình đó	Che tạo ảnh ngoài phân phối; cần nhóm đối chứng che ngẫu nhiên

3.3 Bất định và hiệu chỉnh (Uncertainty and Calibration)

A. Hai loại bất định, và vì sao phải phân biệt

Loại thứ nhất, tiếng Anh gọi là *aleatoric*, là phần mơ hồ nằm trong chính dữ liệu: hai chuyên gia nhìn cùng một ảnh X-quang và bất đồng, hay một vật thể bị che quá nửa. Thu thêm dữ liệu cùng loại không làm nó giảm. Loại thứ hai, tiếng Anh gọi là *epistemic*, là phần do mô hình chưa từng thấy vùng đầu vào này; nó *giảm* khi có thêm dữ liệu đúng chỗ. Phân biệt hai loại là chuyện ngân sách, không phải chuyện học thuật. Nếu bất định ở một nhóm mẫu thuộc loại thứ nhất thì gán nhãn thêm mười nghìn ảnh không đổi được gì. Tiền nên đi chỗ khác: đổi cảm biến, siết lại định nghĩa lớp, hoặc chấp nhận nó và thiết kế đường từ chối trả lời.

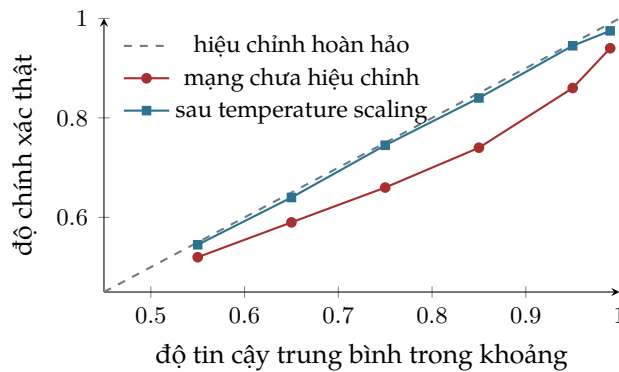
↪ **tiền đề** B/06 — mức đồng thuận giữa người gán nhãn là thước đo trực tiếp của bất định ngẫu nhiên: nó chặn trên mọi mô hình học từ tập nhãn đó

B. Hiệu chỉnh: mạng hiện đại tự tin quá mức

Kết quả đã được đo và lặp lại nhiều lần: các mạng sâu hiện đại cho xác suất cao hơn hẳn độ chính xác thật, và hiện tượng này nặng hơn ở mạng lớn hơn [19]. Cách đo là *ECE*, viết tắt của *expected calibration error*: chia các dự đoán thành các khoảng theo độ tin cậy, rồi lấy trung bình có trọng số của $|\text{độ chính xác} - \text{độ tin cậy}|$ trên từng khoảng,

$$\text{ECE} = \sum_{m=1}^M \frac{|B_m|}{N} |\text{acc}(B_m) - \text{conf}(B_m)|.$$

Nói bằng lời: trong nhóm những lần mô hình nói “tôi chắc 90%”, nó có đúng khoảng 90% số lần không — ECE đo khoảng cách trung bình giữa lời nói và sự thật. Ví dụ tính tay: một mô hình đạt 94% chính xác với độ tin cậy trung bình 0,99 thì riêng khoảng tin cậy cao nhất đã đóng góp $|0,99 - 0,94| = 0,05$; mô hình này “nói quá” năm điểm phần trăm. Hình 3.1 vẽ chính tình huống đó.



Hình 3.1. Biểu đồ độ tin cậy (*reliability diagram*). Đường chấm là chuẩn lý tưởng; đường đỏ nằm dưới nó có nghĩa mô hình nói chắc hơn thực tế ở mọi mức. Temperature scaling kéo đường đỏ về sát đường chuẩn mà *không* đổi thứ hạng giữa các lớp — nên độ chính xác giữ nguyên, chỉ con số xác suất trở nên đúng được. Các điểm ở đây là minh hoạ theo đúng hình dạng đã báo cáo trong tài liệu [19], không phải số đo của một mô hình cụ thể.

C. Bốn cách lấy bất định, và giá của mỗi cách

Bảng 3.2. Bốn công cụ ước lượng bất định. Không cái nào chiếm ưu thế tuyệt đối; chọn theo ngân sách suy luận và theo việc bạn cần bảo đảm loại nào.

Phương pháp	Cơ chế	Chi phí	Hỏng khi nào
Temperature scaling	Chia logit cho một hằng số học trên tập validation	Gần như bằng không	Chỉ sửa hiệu chỉnh tổng thể, không phát hiện mẫu lạ; hỏng khi phân phối dịch chuyển [20]
MC dropout	Lấy mẫu nhiều lần với dropout bật lúc suy luận [21]	k lần forward	Ước lượng phụ thuộc chỗ đặt dropout; thường hẹp hơn bất định thật
Ensemble	Huấn luyện k mô hình độc lập rồi lấy trung bình [22]	k lần huấn luyện và suy luận	Đắt nhất; các thành viên vẫn chia sẻ thiên lệch của dữ liệu
Conformal prediction	Dùng phân vị của điểm bất phù hợp trên tập hiệu chuẩn [23]	Một tập hiệu chuẩn	Bảo đảm là <i>biên</i> , không theo từng lớp; cần tính khả hoán, hỏng dưới dịch chuyển

Trong bốn cách trên, conformal prediction đáng dừng lại vì nó là cách duy nhất cho một *bảo đảm* chứ không chỉ một ước lượng. Ý tưởng rút gọn tới mức đơn giản nhất gồm ba bước. Lấy một tập hiệu chuẩn chưa từng dùng. Với mỗi mẫu, tính một điểm “bất phù hợp”, chẳng hạn $1 - \hat{p}_y$ với $\hat{p}_y$ là xác suất mô hình gán cho nhãn đúng. Rồi lấy phân vị của tập điểm đó làm ngưỡng. Với $n = 1000$ mẫu hiệu chuẩn và mức sai cho phép $\alpha = 0,1$, ngưỡng là điểm lớn thứ $\lceil (n+1)(1-\alpha) \rceil = \lceil 900,9 \rceil = 901$. Khi dự đoán, ta trả về *tập* mọi nhãn có điểm bất phù hợp dưới ngưỡng đó, và tập ấy chứa nhãn đúng với xác suất ít nhất 90%. Điều cần hiểu cho đúng là bảo đảm này mang tính biên: nó đúng trên trung bình toàn bộ phân phối, nên hoàn toàn có thể phủ 99% ở các lớp dễ và 60% ở một lớp khó. Và nó đòi hỏi tính khả hoán giữa tập hiệu chuẩn và dữ liệu triển khai — một giả định mà dữ liệu chuỗi thời gian từ robot vi phạm gần như ngay lập tức, vì các khung hình liên tiếp tương quan mạnh với nhau.

D. Hai hiện tượng nổi vào đây

- **Chưa xác định đủ (*underspecification*)** — nhiều mô hình có cùng điểm test nhưng hành xử rất khác ngoài đời, vì tập test không ràng buộc đủ để chọn ra một trong số chúng. Đây chính là mặt thực nghiệm của điều bạn đã thấy ở chương 24: điểm số bằng nhau không có nghĩa hai hệ thống tương đương.
- **Mẫu đối kháng và ảnh đánh lừa** — vẫn là gradient theo pixel, nhưng đi ngược chiều để phá. Sự tồn tại của chúng nói một điều rất cụ thể: độ tin cậy cao tại một điểm không suy ra độ tin cậy ở lân cận điểm đó. Vì thế phép đo độ bền dưới nhiễu chuẩn hoá [24] phải nằm trong bộ đo mặc định, không phải một thí nghiệm thêm thắt.

CẠM BÃY

Hai cách hiểu sai hay gặp nhất. *Một*, ECE thấp không có nghĩa từng dự đoán đáng tin: một mô hình luôn trả về 0,5 trên một bài toán hai lớp cân bằng có ECE bằng không trong khi không phân biệt được gì cả — hiệu chỉnh là tính chất của *tập* dự đoán, không phải của một dự đoán. *Hai*, ECE phụ thuộc cách chia khoảng: đổi số khoảng thì con số đổi, nên chỉ so ECE giữa hai mô hình khi dùng cùng cách chia. Cả hai đều dẫn tới cùng một kết luận thực hành: đọc biểu đồ độ tin cậy, đừng chỉ đọc một con số ECE.

3.4 Vận hành (Operations)

A. Ngưỡng quyết định đến từ chi phí, không từ cảm tính

Đây là chỗ mọi thứ ở trên biến thành hành động. Với một quyết định nhị phân, ngưỡng tối ưu theo chi phí kỳ vọng là

$$p^* = \frac{c_{FP}}{c_{FP} + c_{FN}},$$

tức là bạn nên báo động khi xác suất hậu nghiệm vượt tỉ lệ giữa chi phí báo động giả và tổng hai chi phí. Câu diễn giải: ngưỡng không phải một siêu tham số để dò, nó là *tỉ số chi phí viết dưới dạng xác suất*.

→ cùng nguyên lý với B/09 — trọng số lớp trong hàm mất mát và ngưỡng quyết định là hai chỗ khác nhau cùng mã hoá một tỉ số chi phí; đặt ở tầng nào là một lựa chọn thiết kế

Ví dụ tính tay: nếu bỏ sót đất gấp một trăm lần báo động giả thì $p^* = 1/101 \approx 0,01$ — ngưỡng thấp đến mức phản trực giác. Nhưng hãy đi tiếp một bước nữa, vì đây là chỗ con số tổng nói dối. Giả sử tỉ lệ dương thật là 1% và ở ngưỡng đó mô hình bắt được 95% ca dương với tỉ lệ báo động giả 10%. Trên mười nghìn mẫu: 100 ca dương thật, bắt được 95; 9900 ca âm, báo nhầm 990. Tức là trong 1085 lần báo động chỉ có 95 lần đúng, độ chính xác của cảnh báo dưới 9%. Ngưỡng ấy *vẫn* là ngưỡng đúng theo chi phí đã khai báo, nhưng nó chỉ khả thi nếu hệ thống có năng lực xử lý hơn một nghìn cảnh báo trên mười nghìn mẫu. Nếu không có năng lực đó thì bài toán thật không phải bài toán ngưỡng mà là bài toán chi phí đã khai báo sai, hoặc bài toán cần một tầng lọc thứ hai.

B. Từ chối trả lời và con người trong vòng lặp

Hệ quả trực tiếp là mọi hệ thống nghiêm túc cần một **đường từ chối trả lời**, và thiết kế nó là phần quan trọng nhất mà ít người học. Ba câu hỏi phải trả lời trước khi triển khai:

1. **Khi nào từ chối?** Một ngưỡng trên xác suất đã hiệu chỉnh, hoặc kích thước tập conformal vượt một giá trị, hoặc một tín hiệu ngoài phân phối [25]. Ngưỡng này phải chọn trên tập validation và phải được báo cáo kèm tỉ lệ từ chối, vì một hệ thống từ chối 40% số ca thì độ chính xác trên phần còn lại là con số vô nghĩa nếu đọc tách rời.
2. **Từ chối rồi thì ai làm tiếp?** Người, một mô hình đắt hơn, hay một hành vi an toàn mặc định. Nếu không có ai làm tiếp thì việc từ chối chỉ chuyển lỗi thành sự im lặng — tệ hơn, vì im lặng không được đếm vào bất kỳ metric nào.
3. **Hành vi an toàn là gì?** Với robot, câu trả lời thường là hạ mức tự chủ: giảm tốc, dừng, hoặc chuyển sang một bộ ước lượng bảo thủ hơn. Bạn đã có sẵn khuôn mẫu này từ SLAM — mất bám thì chuyển sang định vị lại chứ không tiếp tục tích lũy sai số — và nó chuyển sang phía deep learning gần như nguyên vẹn.

→ cùng nguyên lý với C/16 — hạ mức tự chủ khi mất bám trong SLAM và từ chối trả lời khi bất định cao là cùng một mẫu thiết kế: đổi hành vi thay vì đổi ước lượng

C. Giám sát khi không có nhãn

Ở môi trường triển khai bạn gần như không bao giờ có nhãn ngay, nên không đo được độ chính xác. Thứ đo được là các tín hiệu thay thế, chia làm hai nhóm theo giá. Nhóm rẻ tính ngay từ một lần chạy: phân phối độ tin cậy đầu ra, thống kê đặc trưng ở một tầng cố định so với tập tham chiếu, và tỉ lệ từ chối trả lời. Nhóm đắt hơn nhưng nhạy hơn cần thêm hạ tầng: mức bất đồng giữa hai mô hình chạy song song, và các tín hiệu hạ nguồn như người dùng sửa lại kết quả hay robot phải dừng khẩn. Các tín hiệu này phát hiện được *dịch chuyển đầu vào* nhưng gần như mù

với *dịch chuyển khái niệm*. Lý do nằm ở chỗ: khi ảnh trông y hệt mà nhãn đúng đã đổi nghĩa, phân phối đầu vào không nhúc nhích, nên mọi tín hiệu không nhãn đều im lặng. Cách duy nhất là gán nhãn một mẫu nhỏ định kỳ; kinh nghiệm thực hành là ngân sách gán nhãn định kỳ luôn rẻ hơn cái giá phát hiện muộn, dù không có con số phổ quát nào cho “rẻ hơn bao nhiêu”. Vòng khép kín từ lỗi thu được ở production quay về tập dữ liệu chính là chỗ phần E nổi ngược về B/06-chat-luong-du-lieu.

D. Ràng buộc pháp lý là ràng buộc kỹ thuật

Với ảnh mặt người, biển số hay dữ liệu y tế, các ràng buộc về quyền riêng tư quyết định kiến trúc chứ không chỉ quyết định thủ tục giấy tờ: dữ liệu lưu ở đâu, giữ bao lâu, có ra khỏi thiết bị không, có phải xoá theo yêu cầu không. Gắn chúng vào sau khi thiết kế xong thường buộc phải viết lại đúng hai phần đắt nhất — thu thập dữ liệu và giám sát. Kèm theo là nghĩa vụ tài liệu: ghi rõ mô hình được huấn luyện trên dữ liệu nào và dự định dùng trong phạm vi nào [26].

3.5 Giới hạn và trường hợp hỏng

Giới hạn lớn nhất của cả chương: mọi thứ ở đây được hiệu chỉnh và ngưỡng hoá trên một tập hiệu chuẩn, nên chúng chỉ đúng chừng nào dữ liệu triển khai còn giống tập đó. Dưới dịch chuyển phân phối, chất lượng hiệu chỉnh xuống cấp và các phương pháp ước lượng bất định xuống cấp không đồng đều — ensemble thường bền hơn temperature scaling, nhưng không cái nào giữ được bảo đảm ban đầu [20]. Nói cách khác, các công cụ ở chương này quản trị được *bất định đã biết dạng*, không quản trị được cái chưa từng thấy.

Giới hạn thứ hai nằm ở phía diễn giải. Công cụ trực quan hoá không có tiêu chuẩn đúng/sai khách quan, nên rất khó chứng minh một bản đồ nhiệt là sai. Vì thế chúng phải luôn đi kèm can thiệp và đo lại; một bản đồ đứng một mình không được phép là bằng chứng. Giới hạn thứ ba mang tính tổ chức: đường từ chối trả lời chỉ hoạt động nếu thật sự có người và quy trình đón phân bị từ chối. Cách phát hiện sớm là tính trước số ca đẩy sang người mỗi ngày — đúng phép nhân ở mục A — rồi hỏi bộ phận đó xem con số ấy có khả thi không.

CẦN NHỚ

Một ước lượng bất định chỉ có giá trị khi nó nổi vào một *hành động khác*: từ chối trả lời, chuyển cho người, hoặc hạ mức tự chủ. Nếu hệ thống làm y hệt nhau dù bất định cao hay thấp thì bạn đã tổn công tính một con số trang trí. [T2]

3.6 Thực hành

Repo và SOTA. [pytorch/captum](#) (bộ quy gán đầy đủ, có sẵn cả phép kiểm tra tính tảo), [jacobgil/pytorch-grad-cam](#), [lutzroeder/netron](#) để đọc đồ thị mô hình, TensorBoard embedding projector, [ENSTA-U2IS-AI/torch-uncertainty](#), [gpleiss/temperature_scaling](#), [Trusted-AI/adversarial-robustness-toolbox](#) và [bethgelab/foolbox](#), [evidentlyai/evidently](#) cho giám sát drift. Danh sách này chốt tại tháng 9/2026 và sẽ cũ trong sáu đến mười hai tháng; các thư viện hạ tầng bền hơn nhiều so với tên mô hình dẫn đầu bảng.

Câu hỏi kiểm tra hiểu. Mô hình đạt 94% chính xác với độ tin cậy trung bình 0,99 — điều gì sai, đo bằng chỉ số nào, và sửa bằng cách rẻ nhất nào? Grad-CAM chỉ đúng vào vật thể: điều đó chứng minh được gì và *không* chứng minh được gì? Bất định ngẫu nhiên cao ở một nhóm mẫu

— thu thêm dữ liệu có giúp không, và nếu không thì tiêu tiền vào đâu? Vì sao một tập dữ liệu conformal phủ 90% vẫn có thể phủ chỉ 60% ở lớp bạn quan tâm nhất?

Mini project. Chạy phép kiểm tra tính tảo của Adebayo và cộng sự trên một phương pháp quy gán: so bản đồ của mô hình đã huấn luyện với bản đồ của mô hình khởi tạo ngẫu nhiên trên cùng ảnh. Nếu hai bản đồ giống nhau thì ghi lại kết luận đó như một kết quả âm theo đúng chuẩn ở chương 24.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
•• Saliency map	Bản đồ nhiệt gán cho mỗi vùng ảnh một điểm “mức ảnh hưởng tới đầu ra”, tính từ gradient của điểm số theo pixel. Nó dùng để <i>đặt giả thuyết</i> ; nó không đủ để kết luận.
••• Vì sao bản đồ nhiệt đẹp không chứng minh gì?	Vì nhiều phương pháp quy gán vẫn cho bản đồ trông hợp lý ngay cả khi trọng số bị ngẫu nhiên hoá. Khi đó bản đồ phản ánh cấu trúc của ảnh và của chính thuật toán, không phản ánh cái mô hình đã học.
••• Che vùng, đo lại	Phép kiểm chứng thật: che vùng mà bản đồ chỉ vào, đo điểm số tụt bao nhiêu, và so với nhóm đối chứng che một vùng ngẫu nhiên cùng diện tích. Đây là chỗ biến quan sát thành bằng chứng.
• t-SNE	Phép chiếu đặc trưng nhiều chiều xuống hai chiều, bảo toàn lân cận cục bộ. Khoảng cách giữa các cụm và kích thước cụm gần như không mang thông tin, và hình vẽ đổi theo tham số perplexity.
••• Hiệu chỉnh (calibration)	Tính chất rằng trong nhóm những lần mô hình nói “chắc 90%”, nó đúng khoảng 90% số lần. Đây là tính chất của <i>một tập</i> dữ liệu, và nó tách rời hoàn toàn với độ chính xác.
•• ECE	Chia dự đoán thành các khoảng theo độ tin cậy, rồi lấy trung bình có trọng số của khoảng cách giữa độ tin cậy và độ chính xác thật. Nó phụ thuộc cách chia khoảng, nên chỉ so được giữa hai mô hình khi dùng cùng cách chia.
••• Accuracy 94% mà độ tin cậy trung bình 0,99 — sai ở đâu?	Mô hình tự tin quá mức chừng năm điểm phần trăm. Đo bằng ECE và đọc biểu đồ độ tin cậy; sửa rõ nhất bằng temperature scaling, và độ chính xác sẽ không đổi.
•• Temperature scaling	Chia logit cho một hằng số học trên tập hiệu chuẩn. Nó không đổi thứ hạng giữa các lớp nên độ chính xác giữ nguyên; nó cũng không phát hiện mẫu lạ và mất hiệu lực khi phân phối dịch chuyển.
•• Vì sao thêm dữ liệu chỉ giảm được một loại bất định?	Vì chỉ loại do mô hình chưa từng thấy vùng đầu vào đó mới giảm khi có thêm mẫu ở đúng chỗ. Phần mơ hồ nằm trong chính dữ liệu thì không: hai chuyên gia vẫn bất đồng trên cùng tấm ảnh.
•• Ensemble	Huấn luyện k mô hình độc lập rồi lấy trung bình; mức bất đồng giữa chúng là ước lượng bất định. Đắt gấp k lần cả khi huấn luyện lẫn suy luận, nhưng bền nhất dưới dịch chuyển phân phối.
•• Conformal prediction	Trả về một <i>tập</i> nhãn kèm bảo đảm phủ, lấy ngưỡng từ phân vị của tập hiệu chuẩn. Với 1000 mẫu và mức sai 0,1, ngưỡng là điểm bất phù hợp lớn thứ 901.
••• Tập conformal phủ 90% nhưng lớp bạn cần chỉ phủ 60% — vì sao?	Vì bảo đảm là <i>biên</i> : nó đúng trên trung bình toàn phân phối, nên có thể phủ dư ở lớp dễ và thiếu ở lớp khó. Muốn bảo đảm theo lớp thì phải hiệu chuẩn riêng từng lớp và trả giá bằng cỡ tập hiệu chuẩn.
••• Ngưỡng theo chi phí	Ngưỡng báo dương bằng chi phí báo động giả chia cho tổng hai chi phí. Nó là tỉ số chi phí viết dưới dạng xác suất, nên không phải một siêu tham số để dò: bỏ sót đất gấp trăm lần thì ngưỡng khoảng 0,01.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●●● Vì sao ngưỡng đúng vẫn không triển khai được?	Vì tỉ lệ dương thật thấp làm số báo động giả áp đảo. Với 1% dương thật, độ nhạy 95% và tỉ lệ báo động giả 10%, mười nghìn mẫu sinh 1085 cảnh báo mà chỉ 95 cái đúng.
●●● Từ chối trả lời	Nhánh xử lý riêng cho các ca mô hình không đủ chắc, kèm ngưỡng chọn trên tập validation. Nó chỉ hoạt động khi có người hoặc một hành vi an toàn đón phần bị từ chối; không thì nó chỉ đổi lỗi thành im lặng.
●● Drift	Phân phối dữ liệu ở môi trường triển khai trôi khỏi phân phối huấn luyện. Chia hai loại: dịch chuyển đầu vào, khi ảnh đổi; và dịch chuyển khái niệm, khi ảnh trông y hệt nhưng nhãn đúng đã đổi nghĩa.
●●● Độ chính xác ở production tụt nhưng mọi tín hiệu giám sát vẫn bình thường — nghĩ gì?	Hai khả năng: dịch chuyển khái niệm, hoặc một thay đổi ở khâu tiền xử lý mà tín hiệu đang theo dõi không chạm tới. Phân biệt bằng cách gán nhãn một mẫu nhỏ và dựng lại đường dữ liệu từ cảm biến tới tensor đầu vào.
●● Vì sao giám sát không nhãn mù với dịch chuyển khái niệm?	Vì mọi tín hiệu thay thế đều đọc từ đầu vào hoặc từ đầu ra của mô hình. Khi ảnh không đổi mà chỉ nhãn đúng đổi nghĩa, phân phối đầu vào không nhúc nhích, nên phải gán nhãn một mẫu nhỏ định kỳ.

●●● phải thuộc ●● nên nắm ● biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
pytorch/captum	Cài đặt gần như toàn bộ họ quy gán, và quan trọng hơn, cả các phép kiểm tra tính tảo đi kèm	Hạ tầng; là nơi kết luận của Adebayo [18] được đóng gói thành công cụ
Temperature scaling [19]	Bước hậu xử lý một tham số, nay là mặc định trong nhiều hệ thống có ngưỡng quyết định	Rẻ tới mức gần như không có lý do bỏ qua
Deep ensembles [22]	Ước lượng bất định bằng mức bất đồng giữa các mô hình độc lập	Vẫn là chuẩn so sánh mạnh nhất dưới dịch chuyển phân phối [20]
Conformal prediction [23]	Bảo đảm phủ dựa trên phân vị của tập hiệu chuẩn, dùng cho các hệ có ngưỡng và có đường từ chối	Đang lan sang y tế và dự báo; mảng còn mở rộng nhanh
Model cards [26]	Ghi rõ phạm vi sử dụng dự kiến và dữ liệu huấn luyện — phần tài liệu của mục vận hành	Nay là thông lệ ở các nhà phát hành mô hình lớn
evidentlyai/evidently	Giám sát drift bằng các tín hiệu thay thế khi không có nhãn ở production	Hạ tầng; nổi trực tiếp vào vòng lỗi → dữ liệu

MINI PROJECT SLAM · Hiệu chỉnh bộ phân loại đóng vòng và đặt ngưỡng theo chi phí · 1–2 ngày

Mục tiêu	biến điểm số thô của tăng nhận diện lại vị trí (<i>loop closure</i>) thành một xác suất dùng được. Sau đó đặt ngưỡng chấp nhận vòng lặp bằng chi phí lỗi, không bằng cảm tính. Chi phí ở đây lệch hẳn về một phía: một vòng lặp sai kéo cả bản đồ về sai vị trí và hỏng vĩnh viễn, còn một vòng lặp bỏ lỡ chỉ để lại phần trôi vốn đã có.
Dữ liệu	các ứng viên vòng lặp sinh ra khi chạy hệ của bạn trên một chuỗi EuRoC, gán nhãn đúng/sai bằng chân trị tư thế (hai khung hình cách nhau dưới một ngưỡng khoảng cách và góc thì là vòng lặp thật). Vài nghìn ứng viên là đủ; tách một nửa làm tập hiệu chuẩn.
Các bước	vẽ biểu đồ độ tin cậy cho điểm số hiện tại trên tập hiệu chuẩn; học một tham số nhiệt độ rồi vẽ lại để thấy đường cong về sát đường chuẩn; khai báo tỉ số chi phí giữa một

- vòng lặp sai và một vòng lặp bỏ lỡ, tính $p^* = c_{FP} / (c_{FP} + c_{FN})$; đếm số cảnh báo giả ở ngưỡng đó.
- Xong khi** bạn có hai biểu đồ độ tin cậy, một trước và một sau khi hiệu chỉnh. Bạn có một ngưỡng suy ra từ chi phí chứ không chọn tay. Và bạn viết được câu này: “ở ngưỡng đó, trên mười nghìn ứng viên có k vòng lặp sai lọt qua và m vòng lặp thật bị bỏ”, kèm nhận xét hệ thống có chịu nổi k hay không.
- Nối với** E/24 cho việc kiểm tra rằng cải thiện sau hiệu chỉnh vượt sản phẩm bạn đã đo ở mini project chương trước; B/09 cho quan hệ giữa ngưỡng quyết định và trọng số lớp trong hàm mất mát.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì bản đồ có thể gần như không đổi khi trọng số bị ngẫu nhiên hoá, tức là nó phản ánh cấu trúc ảnh và thuật toán quy gán chứ không phản ánh cái mô hình đã học [18]. Thí nghiệm bác bỏ: vẽ cùng bản đồ cho một mô hình khởi tạo ngẫu nhiên và so hai hình.
- A2. Con số sai là 0,99: mô hình nói chắc hơn thực tế khoảng năm điểm phần trăm. Sửa bằng temperature scaling thì *không* làm mô hình chính xác hơn — nó không đổi thứ hạng giữa các lớp — nhưng làm con số xác suất trở nên dùng được để ra quyết định.
- A3. Vì ngưỡng thấp cộng với tỉ lệ dương thật thấp sinh ra rất nhiều báo động giả: với 1% dương thật, độ nhạy 95% và tỉ lệ báo động giả 10%, trên mười nghìn mẫu có 1085 cảnh báo mà chỉ 95 đúng. Ngưỡng vẫn đúng theo chi phí đã khai báo; cái thiếu là năng lực xử lý ngàn ấy cảnh báo.
- A4. Vì bảo đảm của conformal prediction là bảo đảm *biên*: nó đúng trên trung bình toàn bộ phân phối, nên hoàn toàn có thể phủ 99% ở các lớp dễ và 60% ở một lớp khó. Muốn bảo đảm theo từng lớp thì phải dùng biến thể hiệu chuẩn riêng cho từng lớp, và trả giá bằng cỡ tập hiệu chuẩn.

NÓI LẠI THẬT ĐƠN GIẢN

Trong bếp nhà bạn có một cái báo khói, có núm vặn độ nhạy và một dây đèn từ một tới mười cho biết nó “chắc” tới đâu. Cả chương này nói về đúng ba chuyện xảy ra với cái máy ấy.

Chuyện thứ nhất: dây đèn nói dối. Chưa ai từng ngồi đếm xem trong một trăm lần nó bật tới đèn số chín thì thật sự có cháy bao nhiêu lần. Muốn con số ấy dùng được để quyết định, phải đếm rồi chỉnh lại thang đèn cho khớp. Chỉnh xong, cái máy không nhạy hơn chút nào; chỉ có con số trên đèn trở nên đọc được.

Chuyện thứ hai: vặn núm độ nhạy tới đâu không phải chuyện sở thích. Nếu bỏ sót một đám cháy tệ hơn nhiều so với việc nó kêu oan lúc bạn rán trứng, thì vặn rất nhạy là đúng — nhưng phải đếm trước xem mỗi tuần nó sẽ kêu oan bao nhiêu lần, và nhà bạn chịu nổi bấy nhiêu lần không. Máy kêu mà không ai chạy tới thì vặn kiểu gì cũng vô nghĩa.

Chuyện thứ ba: cái đèn chỉ về phía bếp không chứng minh khói đến từ bếp. Muốn biết, đóng cửa bếp lại rồi xem nó còn kêu không.

Cái giá phải trả: mọi thứ bạn chỉnh chỉ đúng cho căn nhà và mùa đã chỉnh; đổi bếp hay sang mùa hanh khô là phải làm lại.

Một câu: Một con số “chắc chín phần mười” chỉ đáng tin khi đã có người ngồi đếm, và chỉ đáng tính khi có ai đó làm khác đi lúc nó báo “không chắc”.

3.7 Kết nối

C/12-co-che-huan-luyen là tiền đề kỹ thuật: cả nhóm trực quan hoá chỉ là gradient đổi biến. B/07-chia-du-lieu-va-danh-gia là nơi đọc ý nghĩa của các con số đi trước hiệu chỉnh — bốn ô của ma trận nhầm lẫn, biểu đồ độ tin cậy, ECE và phân rã điểm Brier. B/08-dich-chuyen-phan-phoi là giới hạn ngoài: hiệu chỉnh và ngưỡng đều mất hiệu lực khi phân phối dịch chuyển, và giám sát drift là cách phát hiện lúc đó. E/24-thi-nghiem-va-ablation

cung cấp kỷ luật đo: cải thiện về hiệu chỉnh cũng phải vượt sà nhiều như mọi cải thiện khác. B/06-chat-luong-du-lieu là nơi vòng lặp khép lại, khi lỗi thu ở production quay về thành dữ liệu mới. D/21-inference-va-trien-khai là mặt song song: nó lo hệ thống có *chạy kịp* không, chương này lo nó có *đáng tin* không.

■ Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao một bản đồ Grad-CAM đẹp trên mô hình khởi tạo ngẫu nhiên lại đủ để bác bỏ giá trị của phương pháp đó, thay vì chỉ làm ta nghi ngờ?
2. Nêu một mô hình có ECE bằng không nhưng hoàn toàn vô dụng. Điều đó nói gì về việc dùng ECE làm metric duy nhất?
3. Temperature scaling không đổi độ chính xác. Vì sao, và vì sao đó vừa là ưu điểm vừa là giới hạn của nó?
4. Chi phí bỏ sót gấp một trăm lần chi phí báo động giả, tỉ lệ dương thật là 1%. Ngưỡng tối ưu là bao nhiêu, và vì sao con số đó có thể khiến hệ thống không triển khai được?
5. Conformal prediction cho bảo đảm gì và *không* cho bảo đảm gì? Giả định nào của nó bị dữ liệu robot vi phạm?
6. Bạn giám sát một mô hình đã triển khai mà không có nhãn. Loại dịch chuyển nào bạn phát hiện được, loại nào không, và bạn bù bằng cách gì?
7. Vì sao một đường từ chối trả lời không có người đón phía sau lại tệ hơn là không có đường từ chối nào?

Nguồn

- [1] S. Keshav, *How to Read a Paper*, ACM SIGCOMM Computer Communication Review, 2007.
- [2] S. Zhang, C. Chi, Y. Yao, et al., “Bridging the Gap Between Anchor-Based and Anchor-Free Detection via Adaptive Training Sample Selection,” in *CVPR*, 2020.
- [3] Z. Liu, H. Mao, C.-Y. Wu, et al., “A ConvNet for the 2020s,” in *CVPR*, 2022.
- [4] K. Musgrave, S. Belongie, and S.-N. Lim, “A Metric Learning Reality Check,” in *ECCV*, 2020.
- [5] M. Ferrari Dacrema, P. Cremonesi, and D. Jannach, “Are We Really Making Much Progress? A Worrying Analysis of Recent Neural Recommendation Approaches,” in *RecSys*, 2019.
- [6] P. Henderson, R. Islam, P. Bachman, et al., “Deep Reinforcement Learning That Matters,” *AAAI*, 2018.
- [7] J. Bergstra and Y. Bengio, “Random Search for Hyper-Parameter Optimization,” in *JMLR*, 2012.
- [8] P. Goyal, P. Dollár, R. Girshick, et al., “Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour,” *arXiv:1706.02677*, 2017.
- [9] C. Zhang, S. Bengio, M. Hardt, et al., “Understanding deep learning requires rethinking generalization,” in *ICLR*, 2017.
- [10] J. Demšar, “Statistical Comparisons of Classifiers over Multiple Data Sets,” *JMLR*, vol. 7, pp. 1–30, 2006.
- [11] B. Recht, R. Roelofs, L. Schmidt, et al., “Do ImageNet Classifiers Generalize to ImageNet?” In *ICML*, 2019.
- [12] C. G. Northcutt, A. Athalye, and J. Mueller, “Pervasive Label Errors in Test Sets Destabilize Machine Learning Benchmarks,” in *NeurIPS Datasets and Benchmarks*, 2021.
- [13] D. Bolya, S. Foley, J. Hays, et al., “TIDE: A General Toolbox for Identifying Object Detection Errors,” in *ECCV*, 2020.
- [14] D. Hoiem, Y. Chodpathumwan, and Q. Dai, “Diagnosing Error in Object Detectors,” in *ECCV*, 2012.
- [15] L. van der Maaten and G. Hinton, “Visualizing Data Using t-SNE,” *JMLR*, vol. 9, pp. 2579–2605, 2008.
- [16] L. McInnes, J. Healy, and J. Melville, “UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction,” *arXiv:1802.03426*, 2018.
- [17] R. R. Selvaraju, M. Cogswell, A. Das, et al., “Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization,” in *ICCV*, 2017.
- [18] J. Adebayo, J. Gilmer, M. Muelly, et al., “Sanity Checks for Saliency Maps,” in *NeurIPS*, 2018.
- [19] C. Guo, G. Pleiss, Y. Sun, et al., “On Calibration of Modern Neural Networks,” in *ICML*, 2017.
- [20] Y. Ovadia, E. Fertig, et al., “Can You Trust Your Model’s Uncertainty?” In *NeurIPS*, 2019.
- [21] Y. Gal and Z. Ghahramani, “Dropout as a Bayesian Approximation: Representing Model Uncertainty in Deep Learning,” in *ICML*, 2016.

- [22] B. Lakshminarayanan, A. Pritzel, and C. Blundell, "Simple and Scalable Predictive Uncertainty Estimation Using Deep Ensembles," in *NeurIPS*, 2017.
- [23] A. N. Angelopoulos and S. Bates, *Conformal Prediction: A Gentle Introduction*. Foundations and Trends in Machine Learning, 2023.
- [24] D. Hendrycks and T. Dietterich, "Benchmarking Neural Network Robustness to Common Corruptions and Perturbations," in *ICLR*, 2019.
- [25] D. Hendrycks and K. Gimpel, "A Baseline for Detecting Misclassified and Out-of-Distribution Examples in Neural Networks," in *ICLR*, 2017.
- [26] M. Mitchell, S. Wu, et al., "Model Cards for Model Reporting," in *FAT**, 2019.

Lộ trình học theo mục tiêu (Learning Paths by Goal)

PHỤ THUỘC không bắt buộc. Đây là chương điều hướng, nên đọc *trước* khi bắt đầu và đọc lại mỗi khi đổi mục tiêu.

TÓM TẮT

Hai mươi lăm chương trước không phải một hàng đợi phải đi hết theo thứ tự. Chương này cất chúng thành sáu lộ trình theo mục tiêu, mỗi lộ trình nói rõ đọc chương nào, theo thứ tự nào, chương nào bỏ được và bỏ thì mất gì. Đọc xong bạn biết mình cần đọc bao nhiêu phần trăm của cây để đạt mục tiêu đang có, thay vì đọc tuần tự rồi bỏ dở ở giữa. Nếu chỉ nhớ một điều: mục tiêu quyết định lộ trình, và lộ trình ngắn nhất tới một mục tiêu cụ thể thường chỉ chiếm một phần ba số chương.

KIẾN THỨC CẦN CÓ TRƯỚC

- **lộ trình (learning path)** — một thứ tự đọc có chủ đích, chọn ra tập chương tối thiểu đủ cho một mục tiêu; khác với mục lục ở chỗ nó *loại bỏ* chương không cần.
- **tầng bán rã [T1]–[T4]** — nhân ở tiêu đề mỗi chương cho biết kiến thức đó sống được bao lâu: [T1] là toán và vật lý, [T4] là công cụ. Khi thời gian eo hẹp, cắt [T4] trước.
- **chương cốt lõi** — chương mà nếu bỏ thì các chương sau biến thành danh sách quy tắc phải học thuộc; trong bảng dưới đánh dấu “bắt buộc”.
- **mini project** — bài tập vài giờ tới hai ngày ở cuối mỗi chương, trong cây này đều gắn với visual SLAM để chúng nối thành một chuỗi liên tục.

CÂU HỎI MỞ ĐẦU

- Q1. Nếu chỉ cần fine-tune một mô hình có sẵn cho một tác vụ hẹp, bao nhiêu phần trăm của cây này là thừa?
- Q2. Vì sao lộ trình “đưa lên thiết bị biên” lại bắt đầu từ một chương ở Phần B chứ không từ Phần D?
- Q3. Vì sao lộ trình dài nhất không phải lộ trình học hết mọi chương?
- Q4. Nếu mục tiêu là áp dụng vào SLAM production, chương nào bạn có thể đọc lướt vì đã biết, và chương nào tuyệt đối không được lướt?

4.1 Đặt vấn đề

Một cây tri thức bốn trăm trang có một chế độ hỏng rất phổ biến: người đọc bắt đầu từ chương một, đi được đến chương bảy hoặc tám, rồi dừng, và phần dừng lại thường là phần họ cần nhất. Nguyên nhân không phải thiếu kỷ luật mà là sai thứ tự. Thứ tự tuyến tính tối ưu cho việc trình bày, không tối ưu cho việc *dùng được sớm*, và một người không thấy kiến thức của mình biến thành thứ chạy được thì sẽ mất động lực trước khi tới chương có giá trị nhất với họ.

Chương này sửa điều đó bằng cách thừa nhận rằng không có một thứ tự đúng. Có sáu mục tiêu thường gặp, và mỗi mục tiêu có một tập chương tối thiểu cùng một thứ tự riêng. Nguyên tắc chung là mọi lộ trình đều phải chạm ba chương nền, còn lại thì cắt theo mục tiêu.

TRỰC GIÁC

Cây này giống một thành phố chứ không giống một con đường. Mục lục là danh sách tên phố; chương này là bản đồ tuyến xe buýt. Cùng một thành phố, người đi làm và người đi du lịch lên hai tuyến khác nhau, và cả hai đều đúng. Sai lầm duy nhất là đi bộ qua từng con phố theo thứ tự bảng chữ cái.

4.2 Ba chương nền, mọi lộ trình đều phải chạm

Trước khi chia nhánh, có ba chương mà mọi lộ trình đều đi qua, vì bỏ chúng thì mọi chương sau mất chỗ bám.

- B/03 — cách đọc một hệ thống lạ. Không có nó thì bạn đọc repo theo tên module thay vì theo giả định, và mọi chương Phần C trở thành danh mục kiến trúc.
- B/04 — nền của việc học từ dữ liệu, và trực đánh đổi giữa giả định và dữ liệu. Đây là trực quay lại nhiều nhất trong cả cây.
- C/12 — cơ chế huấn luyện. Không hiểu vì sao gradient chết thì mọi mô hình là hộp đen, kể cả khi bạn chỉ fine-tune.

4.3 Sáu lộ trình

A. Fine-tune một mô hình có sẵn cho tác vụ hẹp

Mục tiêu: có kết quả dùng được nhanh nhất, chấp nhận chưa hiểu hết bên trong. Đây là lộ trình ngắn nhất, khoảng một phần ba cây.

Bảng 4.1. Lộ trình A — fine-tune. Bảy chương, ước lượng ba tới bốn tuần kể cả mini project.

#	Chương	Vì sao cần	Bỏ thì mất gì
1	B/05	Định nghĩa nhãn và đơn vị phân tích; sai ở đây thì không sửa được ở bước sau	Mô hình học đúng thứ bạn không muốn
2	B/07	Chia dữ liệu và chọn metric; chống rò rỉ	Số đẹp mà vô nghĩa
3	B/03	Đọc được repo và mô hình bạn định dùng	Không biết nó giả định gì
4	C/18	Quyết định scratch, frozen hay fine-tune theo lượng dữ liệu	Chọn sai chế độ, tốn gấp mấy lần
5	C/12	Đủ để chẩn đoán khi loss không giảm	Bé tắc không biết hỏi gì
6	B/09	Loss phải khớp điều bạn thật sự muốn	Tối ưu nhằm mục tiêu
7	E/24	Biết cải thiện là thật hay là nhiều	Tự lừa mình

Bỏ được trong lộ trình này: toàn bộ Phần A trừ khi ảnh đầu vào có vấn đề vật lý, cả C/16 và C/17, và phần lớn Phần D cho tới khi thật sự phải triển khai.

B. Hiểu cơ chế và tự huấn luyện từ đầu

Mục tiêu: không còn hộp đen nào. Lộ trình này nặng nhất về Phần C.

Thứ tự: B/04 → C/12 → D/19 (giai đoạn tự viết bằng NumPy, làm ngay sau chương 12 chứ không đợi tới Phần D) → C/13 → B/09 → B/10 → C/14 → E/24. Phần A đọc theo nhu cầu, mỗi khi một mục ở Phần C cần tới. Ước lượng tám tới mười tuần.

C. Đưa mô hình lên thiết bị biên với ngân sách độ trễ

Lộ trình này bắt đầu từ Phần B chứ không từ Phần D, và đó là điểm dễ gây bất ngờ nhất. Lý do là mọi quyết định nén và tăng tốc chỉ có nghĩa sau khi bạn biết hệ đang nghẽn ở đâu, mà điều đó thuộc B/11.

Thứ tự: B/11 → B/10 → D/20 → D/21 → C/13 (chỉ phần backbone nhẹ) → D/22. Ước lượng ba tới bốn tuần. Bỏ được: C/16, C/17, phần lớn Phần A.

D. Áp dụng vào SLAM production

Đây là lộ trình dài nhất trong sáu lộ trình, nhưng không phải lộ trình đọc hết mọi chương. Nó cắt bỏ mô hình sinh và phần lớn kinh tế học pretraining, đổi lại đi rất sâu vào hình học, chuỗi thời gian và độ tin cậy. Đây cũng là lộ trình duy nhất đi hết Phần F, phần áp dụng học sâu vào chính bài toán SLAM.

Bảng 4.2. Lộ trình D — SLAM production. Năm giai đoạn, ước lượng mười tám tới hai mươi bốn tuần.

Giai đoạn	Chương, theo thứ tự	Điều bạn có được ở cuối giai đoạn
1. Nền	A/02 → B/03 → B/04	Biết ánh vào hệ của bạn đã qua những gì, và biết đọc một repo học sâu như đọc một hệ thống
2. Cơ chế	C/12 → D/19 → C/13 → B/09	Tự huấn luyện và chẩn đoán được một mô hình; hiểu backbone đủ để chọn
3. Miền của bạn	C/15 → C/16 → B/08 → B/11	Nối được flow, tracking, hình học và ngân sách tính toán vào hệ SLAM sẵn có
4. Đưa ra thật	D/20 → D/21 → E/24 → E/25 → D/22	Chạy trên thiết bị trong ngân sách, và chứng minh được cải thiện là thật
5. Áp dụng	F/27 → chọn theo nỗi đau: F/28, F/29, F/30, F/34 → F/31	Quyết định được có nên thay một khối cổ điển bằng mạng hay không, và bảo vệ được quyết định đó bằng số đo

Hai lưu ý riêng cho lộ trình này. Thứ nhất, *đọc lướt được* là A/01 và phần hình học của A/02 nếu bạn đã quen SE(3) và hiệu chỉnh chùm tia, cùng phần ATE và RPE của B/07. Thứ hai, *tuyệt đối không lướt* là B/05 và B/06: định nghĩa nhãn và chất lượng nhãn là chỗ người có nền hình học hay chủ quan nhất, vì trong SLAM cổ điển không có khái niệm nhãn do người gán. Thứ ba, giai đoạn 5 không đọc tuần tự: sau F/27, chọn chương theo nỗi đau đang gặp, và bỏ hẳn F/32 cùng F/33 nếu bài toán của bạn chỉ là quỹ đạo chứ không phải bản đồ dày hay ngữ nghĩa.

E. Đọc paper và làm nghiên cứu

Thứ tự: E/23 → B/03 → E/24 → B/04 → một chương Phần C thuộc lĩnh vực bạn chọn → E/25. Ước lượng bốn tuần, và lộ trình này nên chạy *song song* với mọi lộ trình khác chứ không nối tiếp, vì nó đổi cách bạn đọc mọi chương còn lại.

F. Học toàn bộ

Thứ tự đầy đủ là $A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow F$ theo số chương, với ba ngoại lệ đáng làm khác đi: đọc A/02 kỹ ngay từ đầu nhưng A/01 thì đọc lướt rồi quay lại theo nhu cầu; làm giai đoạn NumPy của D/19 ngay sau C/12; và đọc E/23 cùng E/24 song song với Phần C thay vì để tới cuối. Ước lượng sáu tới chín tháng với nhịp mười giờ mỗi tuần.

4.4 Chuỗi mini project chạy xuyên cây

Mỗi chương kết thúc bằng một mini project gắn với visual SLAM, và chúng được thiết kế để nối thành một chuỗi liên tục thay vì hai mươi lăm bài tập rời. Chuỗi đó thay cho một dự án lớn duy nhất ở cuối, vì một dự án lớn có hai nhược điểm: nó dồn toàn bộ rủi ro vào một lần, và nó chỉ cho phản hồi sau nhiều tuần, tức vi phạm đúng điều kiện phản hồi nhanh mà C/12 và E/24 nói là bắt buộc.

Cách dùng chuỗi này: làm mini project của mọi chương trên lộ trình bạn chọn, bỏ qua mini project của chương bạn đã cắt. Mỗi bài đều có dòng *Xong khi* nêu một tiêu chí đo được, nên bạn biết chắc lúc nào được đi tiếp thay vì tự hỏi đã đủ chưa.

CẠM BẮY

Đừng đọc hết một lộ trình rồi mới bắt đầu làm. Chuỗi mini project chỉ có tác dụng khi mỗi bài được làm ngay sau chương của nó, lúc kiến thức còn tươi và lúc sai lầm còn rẻ. Đọc trước rồi làm sau biến chuỗi thành một dự án lớn duy nhất, tức đúng thứ chương này đang cố tránh.

4.5 Giới hạn và trường hợp hỏng

Sáu lộ trình này là phán đoán về thứ tự phụ thuộc, không phải kết quả đo được; một người có nền khác bạn sẽ cần thứ tự khác. Ước lượng thời gian giả định nhịp mười giờ mỗi tuần và đã có nền kỹ sư phần mềm, nên nó sẽ sai đáng kể với người mới hoàn toàn. Và điều quan trọng nhất: một lộ trình đọc xong không tạo ra năng lực. Chỉ chuỗi mini project mới tạo, còn lộ trình chỉ quyết định bạn làm những bài nào.

CẦN NHỚ

Mục tiêu quyết định lộ trình. Chọn một trong sáu, cắt thẳng tay những chương không nằm trên đó, và làm mini project của từng chương ngay sau khi đọc nó. Ba chương không bao giờ được cắt là B/03, B/04 và C/12.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
●●● Ba chương không bao giờ được cắt	B/03 đọc hệ thống lạ, B/04 nền học từ dữ liệu, C/12 cơ chế huấn luyện. Bỏ một trong ba thì mọi chương sau mất chỗ bám.
●●● Lộ trình	Một thứ tự đọc chọn theo mục tiêu, kèm danh sách chương được phép cắt. Chọn lộ trình là chọn thứ mình <i>không</i> học.
●●● Chuỗi mini project	Bài tập cuối mỗi chương, nối liên tục thay cho một dự án lớn duy nhất ở cuối khoá.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
Vì sao chuỗi thay cho một dự án lớn?	Dự án lớn dồn hết rủi ro vào một lần và chỉ cho phản hồi sau nhiều tuần, tức vi phạm điều kiện phản hồi nhanh mà C/12 và E/24 đòi hỏi.
Vì sao lộ trình biên bắt đầu từ Phần B chứ không Phần D?	Vì mọi kỹ thuật nén và tăng tốc chỉ có nghĩa sau khi biết hệ nghẽn ở đâu, mà chẩn đoán nghẽn nằm ở B/11.
Vì sao lộ trình nghiên cứu chạy song song chứ không nối tiếp?	Vì nó đổi cách bạn đọc mọi chương khác. Để tới cuối thì bạn đã đọc xong bằng thói quen cũ.
Ba ngoại lệ của thứ tự A đến F	Đọc A/02 kỹ ngay nhưng A/01 lướt trước; làm giai đoạn NumPy của D/19 ngay sau C/12; đọc E/23 và E/24 song song với Phần C.
Giai đoạn 5 của lộ trình SLAM	Phần F, và nó không đọc tuần tự: sau F/27 thì chọn chương theo nỗi đau đang gặp.
Hai chương của Phần F bỏ được nếu chỉ cần quỹ đạo	F/32 bản đồ neural và F/33 ngữ nghĩa. Chúng phục vụ bản đồ dày và nhiệm vụ, không phục vụ độ chính xác quỹ đạo.
Dòng <i>Xong khi</i>	Tiêu chí đo được của mỗi mini project, để biết lúc nào được đi tiếp thay vì tự hỏi đã đủ chưa.
Đọc hết một lộ trình rồi mới làm — hỏng ở đâu?	Nó biến chuỗi mini project trở lại thành một dự án lớn duy nhất, đúng thứ chương này cố tránh.
Ước lượng thời gian ở đây giá định gì?	Nhịp mười giờ mỗi tuần và đã có nền kỹ sư phần mềm. Sai đáng kể với người mới hoàn toàn.

- phải thuộc
- nên nắm
- biết là có

MINI PROJECT SLAM · Viết lộ trình của chính bạn thành lịch chạy được · 1–2 ngày, rồi chạy tuần đầu

Mục tiêu	Biến sáu lộ trình trừu tượng thành một lịch có ngày tháng, để tuần sau bạn mở ra là biết đọc chương nào và làm bài nào, không phải quyết định lại.
Dữ liệu	Không cần dữ liệu. Cần danh sách chương của cây, quỹ thời gian thật mỗi tuần của bạn, và một câu phát biểu mục tiêu trong một dòng.
Các bước	Chọn một trong sáu lộ trình theo mục tiêu đó. Liệt kê các chương của nó và <i>gạch tên</i> những chương bị cắt — viết ra danh sách bị cắt quan trọng ngang danh sách phải đọc. Chia theo tuần với nhịp thật của bạn chứ không nhịp lý tưởng. Với mỗi tuần, ghi kèm mini project của chương đó và dòng <i>Xong khi</i> của nó. Cuối cùng đặt một mốc rà lại sau bốn tuần.
Xong khi	Bạn có một bảng gồm tuần, chương, mini project, tiêu chí xong; và bạn đã hoàn thành mini project của tuần đầu tiên đúng theo tiêu chí đó.
Nối với	E/24 cho cách đo tiến độ mà không tự lừa mình; B/03 là chương đầu tiên của mọi lộ trình.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Khoảng hai phần ba. Lộ trình A chỉ cần bảy chương; toàn bộ Phần A, hai chương 3D và mô hình sinh, cùng phần lớn Phần D đều bỏ được cho tới khi thật sự triển khai.
- A2. Vì mọi kỹ thuật nén và tăng tốc chỉ có nghĩa sau khi biết hệ nghẽn ở đâu, mà chẩn đoán nghẽn thuộc B/11. Bắt đầu từ Phần D thì bạn chọn kỹ thuật trước khi biết bệnh.
- A3. Vì lộ trình dài nhất là lộ trình SLAM production, và nó vẫn cắt bỏ mô hình sinh cùng phần lớn kinh tế học pretraining. Học hết mọi chương là lộ trình *rộng* nhất, không phải *sâu* nhất theo một mục tiêu.
- A4. Lướt được phần toán ở A/01, phần hình học nhiều view ở A/02, và phần ATE cùng RPE ở B/07. Không được lướt B/05 và B/06, vì nhân do người gán là khái niệm không tồn tại trong SLAM cổ điển.

NÓI LẠI THẬT ĐƠN GIẢN

Hình dung bạn có bản đồ một thành phố và mười giờ rảnh mỗi tuần. Bản đồ vẽ mọi con đường, nhưng bạn không đi hết được, và cũng không cần. Việc đầu tiên không phải chọn đường nào để đi mà là nói ra bạn muốn đến đâu; khi đã nói được điều đó thì phần lớn con đường tự loại mình. Chương này làm đúng ba việc. Nó hỏi bạn muốn đến đâu, và ép câu trả lời gói trong một dòng. Nó đưa sáu con đường đã có người đi, kèm danh sách những khu phố bạn được quyền không ghé. Và nó nhắc rằng đi bộ mới tính là đi: mỗi chương có một bài tập nhỏ, làm ngay sau khi đọc, vì kiến thức còn tươi và sai lầm còn rẻ.

Một câu: Chọn lộ trình thực chất là chọn thứ mình sẽ không học, và đó mới là quyết định khó.

Chỗ không rút gọn được: Vì sao ba chương nền lại đúng là ba chương đó, chứ không phải ba chương khác — đây là phán đoán về thứ tự phụ thuộc, chưa ai đo được.

4.6 Kết nối

Chương này là bản đồ của toàn cây, nên nó nối tới mọi chương. Hai liên kết đáng nhớ nhất: [E/24](#) giải thích vì sao mọi lộ trình đều kết thúc bằng chương thực nghiệm, và [B/03](#) là chương duy nhất xuất hiện trong cả sáu lộ trình.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Mục tiêu của bạn, viết trong một dòng, dẫn tới lộ trình nào trong sáu lộ trình?
2. Ba chương nào không bao giờ được cắt, và vì sao cắt một trong ba lại làm hỏng mọi chương sau?
3. Lộ trình đưa mô hình lên thiết bị biên bắt đầu từ Phần B chứ không Phần D — lý do là gì?
4. Vì sao chuỗi mini project thay cho một dự án lớn duy nhất ở cuối, và hai nhược điểm của dự án lớn là gì?
5. Giai đoạn 5 của lộ trình SLAM không đọc tuần tự; bạn chọn chương của Phần F theo tiêu chí nào?
6. Nếu bạn đọc hết một lộ trình trước rồi mới làm bài tập, điều gì hỏng?

Nguồn

- [1] S. Keshav, *How to Read a Paper*, ACM SIGCOMM Computer Communication Review, 2007.
- [2] S. Zhang, C. Chi, Y. Yao, et al., “Bridging the Gap Between Anchor-Based and Anchor-Free Detection via Adaptive Training Sample Selection,” in *CVPR*, 2020.
- [3] Z. Liu, H. Mao, C.-Y. Wu, et al., “A ConvNet for the 2020s,” in *CVPR*, 2022.
- [4] K. Musgrave, S. Belongie, and S.-N. Lim, “A Metric Learning Reality Check,” in *ECCV*, 2020.
- [5] M. Ferrari Dacrema, P. Cremonesi, and D. Jannach, “Are We Really Making Much Progress? A Worrying Analysis of Recent Neural Recommendation Approaches,” in *RecSys*, 2019.
- [6] P. Henderson, R. Islam, P. Bachman, et al., “Deep Reinforcement Learning That Matters,” *AAAI*, 2018.
- [7] J. Bergstra and Y. Bengio, “Random Search for Hyper-Parameter Optimization,” in *JMLR*, 2012.
- [8] P. Goyal, P. Dollár, R. Girshick, et al., “Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour,” *arXiv:1706.02677*, 2017.
- [9] C. Zhang, S. Bengio, M. Hardt, et al., “Understanding deep learning requires rethinking generalization,” in *ICLR*, 2017.
- [10] J. Demšar, “Statistical Comparisons of Classifiers over Multiple Data Sets,” *JMLR*, vol. 7, pp. 1–30, 2006.
- [11] B. Recht, R. Roelofs, L. Schmidt, et al., “Do ImageNet Classifiers Generalize to ImageNet?” In *ICML*, 2019.
- [12] C. G. Northcutt, A. Athalye, and J. Mueller, “Pervasive Label Errors in Test Sets Destabilize Machine Learning Benchmarks,” in *NeurIPS Datasets and Benchmarks*, 2021.
- [13] D. Bolya, S. Foley, J. Hays, et al., “TIDE: A General Toolbox for Identifying Object Detection Errors,” in *ECCV*, 2020.
- [14] D. Hoiem, Y. Chodpathumwan, and Q. Dai, “Diagnosing Error in Object Detectors,” in *ECCV*, 2012.
- [15] L. van der Maaten and G. Hinton, “Visualizing Data Using t-SNE,” *JMLR*, vol. 9, pp. 2579–2605, 2008.
- [16] L. McInnes, J. Healy, and J. Melville, “UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction,” *arXiv:1802.03426*, 2018.
- [17] R. R. Selvaraju, M. Cogswell, A. Das, et al., “Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization,” in *ICCV*, 2017.
- [18] J. Adebayo, J. Gilmer, M. Muehly, et al., “Sanity Checks for Saliency Maps,” in *NeurIPS*, 2018.
- [19] C. Guo, G. Pleiss, Y. Sun, et al., “On Calibration of Modern Neural Networks,” in *ICML*, 2017.
- [20] Y. Ovadia, E. Fertig, et al., “Can You Trust Your Model’s Uncertainty?” In *NeurIPS*, 2019.
- [21] Y. Gal and Z. Ghahramani, “Dropout as a Bayesian Approximation: Representing Model Uncertainty in Deep Learning,” in *ICML*, 2016.

- [22] B. Lakshminarayanan, A. Pritzel, and C. Blundell, “Simple and Scalable Predictive Uncertainty Estimation Using Deep Ensembles,” in *NeurIPS*, 2017.
- [23] A. N. Angelopoulos and S. Bates, *Conformal Prediction: A Gentle Introduction*. Foundations and Trends in Machine Learning, 2023.
- [24] D. Hendrycks and T. Dietterich, “Benchmarking Neural Network Robustness to Common Corruptions and Perturbations,” in *ICLR*, 2019.
- [25] D. Hendrycks and K. Gimpel, “A Baseline for Detecting Misclassified and Out-of-Distribution Examples in Neural Networks,” in *ICLR*, 2017.
- [26] M. Mitchell, S. Wu, et al., “Model Cards for Model Reporting,” in *FAT**, 2019.